

**BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)
COLLEGE OF ENGINEERING, PUNE- 4110043**

**PROJECT BASED LEARNING
REPORT ON**

**WHEAT LEAF DISEASE DETECTION USING
RESNET50**

Submitted by

Name	PRN	Roll no.
Yash Patil	2314110168	67
Rohit Patil	2314110167	66
Adarsh Kumar	2314110173	72
Shivam Soni	2314110194	90

**For the course
Data Warehousing and Mining
Submitted To
Prof. Rohini Khalkar**

**B.Tech Computer (Sem V)
2025-26**



DEPARTMENT OF COMPUTER ENGINEERING

Bharati Vidyapeeth (Deemed To Be University)

College of Engineering Pune – 411043



CERTIFICATE

This is to certify that the PROJECT BASED LEARNING work entitled “Wheat Leaf Disease Detection Using RESNET50” is a project-based learning work carried out under the course coordinator of (Data Warehousing and Mining) Prof. Rohini Khalkar and it is submitted towards the partial fulfilment of the requirement of Bharati Vidyapeeth (Deemed to be University) College of Engineering, Pune. For this project awarded of marks on the internal assessment test for the Bachelor of Technology (Computer Engineering).

Date:

Prof. Rohini Khalkar
(Course Coordinator)

H.O.D
Computer Engineering

ACKNOWLEDGEMENT

We would like to extend our sincere gratitude to the HOD Prof. Sandeep Vanjale for nurturing a congenial yet competitive environment, which motivates all the students not only to pursue goals but also to elevate the Humanitarian level. We would like to thank our college and department for helping us to select our project. We would like to express our gratitude and appreciation to all those who gave us the opportunity to complete this report.

Inspiration and guidance are invaluable in every aspect of life, which we have received from our respected course coordinator **Prof. Rohini Khalkar** who gave us her careful and ardent guidance and because of which we were able to complete ~~is~~project. Any number of words won't suffice to express our gratitude to her untiring devotion.

We would also like to acknowledge the professors of our department for their valuable support and continuous guidance.

TABLE OF CONTENTS

Sr. No.	Topic	Page No.
1	Introduction	1
	1.1 Background on Wheat Diseases and Agricultural Impact	1
	1.2 Problem Statement and Motivation	2
	1.3 Project Scope and Objectives	3
2	System Architecture	4
	2.1 System Architecture and Design	4
	2.2 Data Flow Architecture	4
	2.3 Technology Stack	5
3	Methodology	6
	3.1 Dataset Description	6
	3.2 Image Processing Techniques	7
	3.3 Data Augmentation Strategies	7
	3.4 Model Architecture	8
	3.5 Layer Specifications and Parameters	10
	3.6 Training Process	10
	3.7 Validation Strategy	11
	3.8 Web Application Development	11
	3.9 User Interface Design	12
4	Implementation Details	13
	4.1 Frontend Implementation	14
	4.2 Backend Implementation	14
	4.3 Deep Learning Model (Resnet50)	15
5	Testing Techniques and Test Plan	16
	5.1 Unit Testing	16
	5.2 Functional Testing	17
	5.3 Model Evaluation	18
	5.4 Integration Testing	19
	5.5 User Interface and User Experience Testing	20
6	Code	20-31
7	Output	32-43
8	Performance Analysis	44
	8.1 Model Performance	45
	8.2 Performance Report	46
9	Summary and Conclusion	47
10	Future Scope	48
11	References	49

1.INTRODUCTION

1.1 Background on Wheat Diseases and Agricultural Impact

Wheat (*Triticum aestivum*) stands as one of the world's most crucial cereal crops, serving as a primary food source for over 35% of the global population and contributing significantly to global food security (FAO, 2021). As the second most-produced cereal crop globally, wheat cultivation spans across diverse climatic zones, from temperate regions to subtropical areas, making it vulnerable to a wide array of diseases that can severely impact yield and quality. Plant diseases represent one of the most significant threats to agricultural productivity, with wheat diseases alone causing estimated annual losses of 10-15% of global wheat production (Savary et al., 2019). Among the most devastating wheat diseases are Fusarium Head Blight (FHB), Leaf Rust, and Tan Spot, each presenting unique challenges to farmers and agricultural systems worldwide. Fusarium Head Blight, caused primarily by *Fusarium graminearum*, not only reduces grain yield by 20-50% but also produces mycotoxins that contaminate grain, making it unsuitable for human and animal consumption (McMullen et al., 2012). Leaf Rust, caused by *Puccinia trichina*, can cause yield losses of up to 40% under favourable conditions and requires constant monitoring due to its rapid spread capabilities (Kolmer, 2013). Tan Spot, caused by *Pyrenophora tritici-repentis*, affects both yield and grain quality, with losses ranging from 5-30% depending on environmental conditions and cultivar susceptibility (Lamari et al., 2003). The economic impact of these diseases extends beyond direct yield losses, encompassing increased production costs for fungicide applications, reduced grain quality affecting market prices, and the need for resistant cultivar development. Traditional disease management strategies rely heavily on visual inspection by agricultural experts, which is time-consuming, subjective, and often requires specialized knowledge that may not be readily available in all farming communities.

1.2 Problem Statement and Motivation

The current approach to wheat disease identification and management faces several critical limitations that hinder effective agricultural decision-making and crop protection strategies. Traditional methods of disease detection rely primarily on manual visual inspection by farmers or agricultural extension workers, a process that is inherently subjective, time-intensive, and requires extensive expertise in plant pathology. The challenges associated with conventional disease identification methods include:

Expertise Dependency: Accurate disease identification requires specialized knowledge of plant pathology, symptom recognition, and disease progression patterns. This expertise is not uniformly distributed across all agricultural regions, particularly in developing countries where access to trained agricultural professionals may be limited.

Time Sensitivity: Many wheat diseases progress rapidly, and early detection is crucial for effective management. Manual inspection methods often result in delayed identification, allowing diseases to spread and cause more significant damage before intervention measures can be implemented.

Subjectivity and Inconsistency: Visual inspection results can vary significantly between different observers, leading to inconsistent diagnoses and potentially inappropriate treatment recommendations.

Scalability Limitations: Manual inspection methods are not suitable for large-scale monitoring programs or precision agriculture applications where rapid, automated assessment of multiple fields or plants is required.

Cost Implications: The labour-intensive nature of manual disease identification increases operational costs for farmers and agricultural organizations, particularly when frequent monitoring is necessary. The rapid advancement of artificial intelligence and computer vision technologies presents unprecedented opportunities to address these challenges through automated disease detection systems. Machine learning algorithms, particularly deep learning models, have demonstrated remarkable capabilities in image recognition tasks, offering the potential to revolutionize agricultural disease management through accurate, rapid, and scalable identification systems.

1.3 Project Scope and Objectives

This project aims to develop an automated wheat disease prediction system that leverages state-of-the-art deep learning techniques to accurately identify and classify common wheat diseases from digital images. The system is designed to provide farmers, agricultural professionals, and researchers with a reliable, accessible, and user-friendly tool for rapid disease identification and assessment.

Primary Objectives

1. Disease Classification Accuracy: Develop a machine learning model capable of accurately classifying wheat diseases with high precision and recall rates, specifically targeting Fusarium Head Blight, Leaf Rust, Tan Spot, and healthy wheat conditions.

2. User-Friendly Interface: Create an intuitive web-based application that allows users to upload wheat leaf images and receive immediate, accurate disease predictions with confidence scores.

3. Transfer Learning Implementation: Utilize pre-trained deep learning models, specifically ResNet50, to achieve optimal performance while minimizing computational requirements and training time.

4. Real-World Applicability: Ensure the system's practical utility by developing a robust, scalable solution that can be deployed across different agricultural environments and user skill levels.

Secondary Objectives

1. Performance Optimization: Optimize model performance through comprehensive hyperparameter tuning, data augmentation techniques, and advanced training strategies to achieve state-of-the-art classification accuracy.

2. System Reliability: Implement robust error handling, input validation, and quality assurance measures to ensure consistent and reliable system performance across diverse input conditions.

3. Educational Value: Provide detailed documentation and analysis of the machine learning pipeline to serve as a reference for future agricultural technology development and academic research.

4. Scalability Considerations: Design the system architecture to support future enhancements, including additional disease types, improved model architecture, and integration with larger agricultural management systems.

2.SYSTEM ARCHITECTURE

2.1 System Architecture and Design:

The wheat disease detection system follows a three-tier architecture comprising:

1. Presentation Layer (Web Interface)
2. Application Layer (Flask Backend + ML Pipeline)
3. Data Layer (Model Files + Image Storage)

System Components:

- **Frontend:** HTML/CSS/JavaScript web interface
- **Backend:** Flask web server
- **ML Engine:** ResNet50-based disease classifier
- **Storage:** Static file storage for uploads and model persistence

2.2 Data Flow Architecture

1. User uploads wheat leaf image via web interface
2. Flask receives image through POST request
3. Image preprocessing pipeline (resize, normalize, validate)
4. ResNet50 model performs inference
5. Prediction results formatted and returned
6. Web interface displays disease classification and confidence

Wheat Disease Detection Data Flow

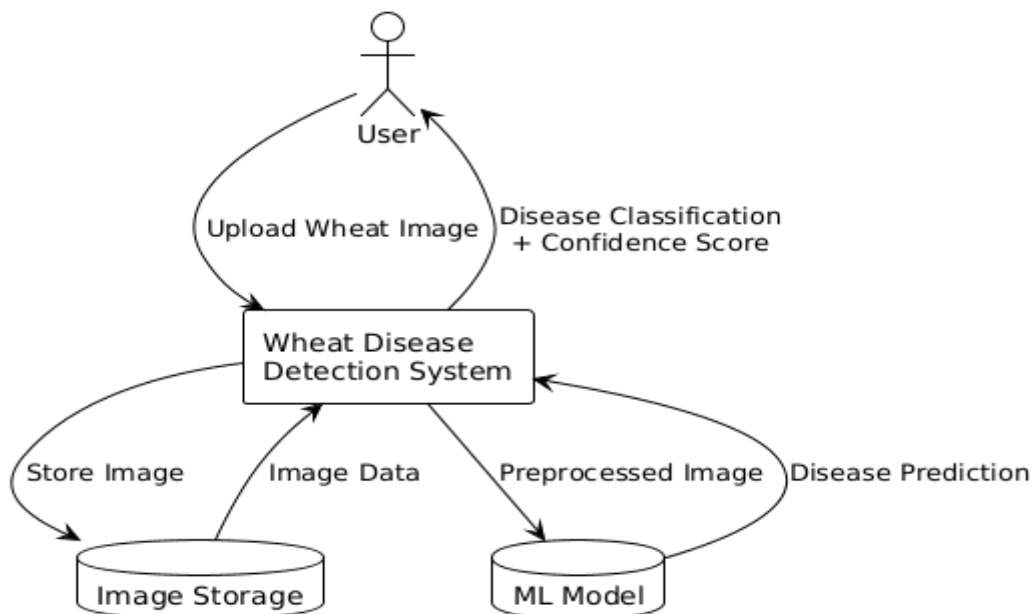


Figure 1. Data Flow Diagram

2.3 Technology Stack:

Backend Technologies:

- **Framework:** Flask (Python web framework)
- **ML Library:** TensorFlow/Keras for deep learning
- **Image Processing:** OpenCV for image manipulation
- **Model Architecture:** ResNet50 with transfer learning

Frontend Technologies:

- **Markup:** HTML5 with semantic structure
- **Styling:** CSS3 with custom variables and responsive design
- **Interactivity:** JavaScript for file handling and preview

Data Management:

- **Model Storage:** HDF5 format (.h5) for trained model
- **Label Encoding:** Pickle format (.pickle) for class mappings
- **Image Storage:** Static directory for temporary uploads

3.METHODOLOGY

3.1 Dataset Description

The wheat disease prediction system was developed using a comprehensive dataset comprising digital images of wheat leaves exhibiting various disease conditions. The dataset was organized into four distinct categories, each representing a specific wheat condition: Fusarium Head Blight, Healthy Wheat, Leaf Rust, Tan Spot and Unknown images to handle outlier images.

Dataset Structure and Distribution:

Fusarium Head Blight: 1,057 images (984 JPG, 36 JPEG, 31 PNG, 6 other formats)

Healthy Wheat: 1,280 images (1,100 JPG, 134 JFIF, 38 JPEG, 8 other formats)

Leaf Rust: 1,622 images (1,058 JPG, 332 JFIF, 138 PNG, 94 other formats)

Tan Spot: 1,033 images (922 JPG, 57 JPEG, 42 PNG, 12 other formats)

Unknown: 1,875 images (JPG format)

Total Dataset Size: 6,799 images across all categories, providing a substantial foundation for training a robust deep learning model. The dataset exhibits a balanced distribution across disease categories, with Leaf Rust representing the largest subset (32.5%) and Tan Spot the smallest (20.7%). This distribution reflects the relative prevalence and photographic availability of different wheat disease conditions in agricultural settings.

Image Characteristics:

The dataset encompasses images captured under diverse conditions, including varying lighting conditions, camera angles, image resolutions, and background environments. This diversity enhances the model's generalization capabilities and ensures robust performance across different real-world scenarios. Images range from close-up leaf examinations to broader field views, providing comprehensive coverage of disease manifestation patterns.

3.2 Image Preprocessing Techniques

The preprocessing pipeline was designed to standardize input images while preserving essential disease-related features. The preprocessing steps were carefully selected to optimize model performance while maintaining computational efficiency.

Image Loading and Format Standardization:

All images were loaded using OpenCV (cv2) library, which provides robust handling of various image formats including JPG, JPEG, PNG, and JFIF. The loading process included automatic format detection and conversion to ensure consistent processing regardless of the original file format.

Color Space Conversion:

Images were converted from the default BGR (Blue-Green-Red) color space used by OpenCV to RGB (Red-Green-Blue) color space, which is the standard format expected by pre-trained models and ensures compatibility with the ResNet50 architecture.

Image Resizing:

All images were resized to 224×224 pixels, which is the standard input size for ResNet50 models. This resizing was performed using OpenCV's resize function with default interpolation, maintaining aspect ratio considerations while ensuring uniform input dimensions across the entire dataset.

Normalization Process:

Image normalization was implemented using mean subtraction with values derived from ImageNet dataset preprocessing standards:

- Red channel: 123.68
- Green channel: 116.779
- Blue channel: 103.939

This normalization approach aligns with the preprocessing used during ResNet50's original training on ImageNet, ensuring optimal transfer learning performance by maintaining consistency with the pre-trained model's expected input distribution.

Data Type Conversion:

Images were converted to float32 data type to ensure numerical precision during model training and inference, while maintaining computational efficiency.

3.3 Data Augmentation Strategies

Data augmentation was implemented to increase dataset diversity, improve model generalization, and prevent overfitting. The augmentation strategy was designed to simulate real-world variations in image capture conditions while preserving disease-related visual features.

Training Augmentation Pipeline:

The training data augmentation included the following transformations:

- **Rotation Range:** ± 30 degrees to simulate different camera angles and leaf orientations
- **Zoom Range:** 0.15 scaling factor to account for varying distances between camera and subject
- **Width Shift Range:** 0.2 horizontal translation to simulate different framing approaches
- **Height Shift Range:** 0.2 vertical translation for comprehensive spatial augmentation
- **Shear Range:** 0.15 degrees to simulate perspective variations

- **Horizontal Flip:** Enabled to double the effective dataset size and improve symmetry handling
- **Fill Mode:** "nearest" interpolation to handle edge cases during transformations

Validation Augmentation:

Validation data underwent minimal preprocessing, including only mean subtraction normalization, to ensure unbiased evaluation of model performance on unmodified test data.

Augmentation Rationale:

The selected augmentation parameters were chosen to reflect realistic variations encountered in agricultural image capture scenarios while avoiding transformations that could potentially distort disease-related visual features. The augmentation intensity was balanced to provide sufficient diversity without introducing artifacts that could mislead the model during training.

3.4 Model Architecture

ResNet50 Transfer Learning Approach

The model architecture leverages ResNet50, a deep residual network that has demonstrated exceptional performance in image classification tasks. ResNet50's architecture is particularly well-suited for transfer learning applications due to its robust feature extraction capabilities and proven effectiveness across diverse domains.

ResNet50 Architecture Overview:

ResNet50 consists of 50 layers organized into five main blocks, each containing multiple residual connections that enable effective gradient flow during training. The architecture includes:

- **Input Layer:** Accepts $224 \times 224 \times 3$ RGB images
- **Initial Convolution Block:** 7×7 convolution with 64 filters, followed by batch normalization and ReLU activation
- **Residual Blocks:** Four main blocks containing 3, 4, 6, and 3 residual units respectively
- **Global Average Pooling:** Reduces spatial dimensions while preserving channel information
- **Classification Head:** Fully connected layers for final prediction

Transfer Learning Implementation:

The transfer learning approach utilized ResNet50 pre-trained on ImageNet dataset, which provides a strong foundation of general visual feature extraction capabilities. The pre-trained weights were frozen for the base ResNet50 layers, allowing the model to leverage learned low-level and mid-level features while focusing training efforts on task-specific high-level features.

Custom Classification Head:

A custom classification head was designed specifically for the four-class wheat disease classification task:

- **Global Average Pooling:** Converts spatial feature maps to fixed-size vectors
- **Flatten Layer:** Ensures proper tensor reshaping for dense layer processing
- **Dense Layer:** 512 neurons with ReLU activation for feature transformation
- **Dropout Layer:** 0.4 dropout rate to prevent overfitting
- **Output Layer:** 4 neurons with softmax activation for final classification

Wheat Disease Prediction Model Architecture

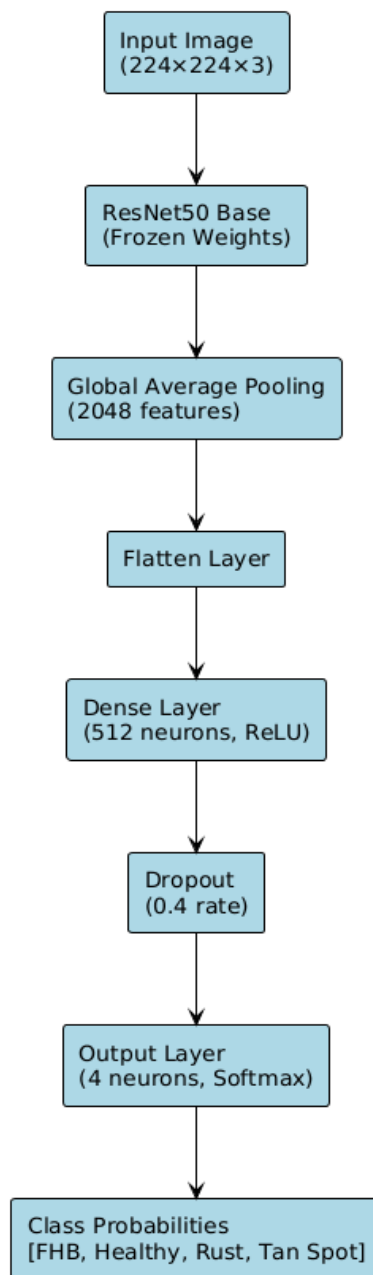


Figure 2. Model Architecture Diagram

3.5 Layer Specifications and Parameters

The frozen ResNet50 base includes:

- **Convolutional Layers:** Multiple 3×3 and 1×1 convolutions with batch normalization
- **Residual Connections:** Skip connections enabling deep network training
- **Pooling Layers:** Max pooling for spatial dimension reduction
- **Activation Functions:** ReLU activations throughout the network

Custom Classification Head Specifications:

- **Global Average Pooling:** Reduces $7 \times 7 \times 2048$ feature maps to 2048-dimensional vectors
- **Dense Layer:** 512 neurons with ReLU activation and L2 regularization
- **Dropout:** 0.4 dropout rate applied during training to prevent overfitting
- **Output Layer:** 4 neurons with softmax activation for probability distribution

Weight Initialization:

The custom classification head weights were initialized using Xavier/Glorot initialization, ensuring proper gradient flow during training. The frozen ResNet50 weights retained their original ImageNet-trained values.

Optimization Strategy:

The Adam optimizer was chosen for its adaptive learning rate capabilities and proven effectiveness in deep learning applications. Adam combines the benefits of momentum-based optimization with adaptive learning rates, providing robust convergence across diverse optimization landscapes.

3.6 Training Process

The training process was carefully designed to optimize model performance while ensuring stable convergence and preventing overfitting. The hyperparameter selection was based on empirical testing and established best practices for transfer learning applications.

Optimizer Configuration:

- **Optimizer:** Adam optimizer with adaptive learning rates
- **Initial Learning Rate:** $1e-3$ (0.001)
- **Beta Parameters:** $\beta_1=0.9$, $\beta_2=0.999$ (default Adam parameters)

Training Configuration:

- **Batch Size:** 64 images per batch
- **Epochs:** Maximum 20 epochs with early stopping
- **Validation Split:** 25% of data reserved for validation
- **Stratified Split:** Ensures balanced representation across all classes

3.7 Validation Strategy

Data Splitting Approach:

The dataset was split using stratified sampling to ensure balanced representation across all disease categories in both training and validation sets. The 75/25 split ratio provides sufficient training data while maintaining a robust validation set for performance evaluation.

Validation Metrics:

- **Accuracy:** Overall classification accuracy
- **Loss:** Validation loss for convergence monitoring
- **Per-Class Metrics:** Precision, recall, and F1-score for each disease category

Early Stopping Implementation:

Early stopping was configured with the following parameters:

- **Monitor:** Validation accuracy
- **Patience:** 10 epochs
- **Mode:** Maximum (stops when validation accuracy stops improving)
- **Restore Best Weights:** Enabled to retain the best performing model

Model Checkpointing:

Model checkpointing was implemented to save the best performing model based on validation accuracy:

- **Monitor:** Validation accuracy
- **Save Best Only:** Enabled to conserve storage space
- **Mode:** Maximum (saves model with highest validation accuracy)

3.8 Web Application Development

Flask Framework Implementation:

The web application was developed using Flask, a lightweight and flexible Python web framework that provides excellent support for machine learning model deployment and integration.

Application Architecture:

The Flask application follows a modular design pattern with clear separation of concerns:

- **Model Loading:** Pre-trained model and label binarizer loaded at application startup
- **Image Processing:** Dedicated function for image preprocessing and prediction
- **Route Handling:** Clean separation between home page and prediction endpoints
- **Error Handling:** Comprehensive error handling for various failure scenarios

Core Components:

- **Model Initialization:** Loads the trained ResNet50 model and label binarizer at startup
- **Prediction Pipeline:** Handles image upload, preprocessing, and model inference

- **Response Generation:** Formats prediction results for web display
- **Static File Management:** Handles uploaded image storage and serving

3.9 User Interface Design

Design Philosophy:

The user interface was designed with accessibility and usability as primary considerations, ensuring that users with varying technical expertise can effectively utilize the system.

Visual Design Elements:

- **Color Scheme:** Green-based palette reflecting agricultural themes
- **Typography:** Clean, readable fonts with appropriate sizing for different screen sizes
- **Layout:** Responsive design that adapts to various device screen sizes
- **Visual Feedback:** Clear indication of prediction confidence levels

User Experience Features:

- **Drag-and-Drop Upload:** Intuitive file selection interface
- **Image Preview:** Real-time preview of uploaded images
- **Confidence Indicators:** Visual representation of prediction confidence
- **Error Messages:** Clear, actionable error messages for various failure scenarios

Deployment Considerations

System Requirements:

- **Python Environment:** Python 3.7+ with required dependencies
- **Web Server:** Flask development server or production WSGI server
- **Storage:** Sufficient disk space for model files and uploaded images
- **Memory:** Adequate RAM for model loading and inference operations

Security Considerations:

- **File Upload Validation:** Restriction to image file types only
- **Input Sanitization:** Validation of uploaded file content
- **Error Handling:** Secure error messages that don't expose system internals

Performance Optimization:

- **Model Caching:** Pre-loaded model eliminates startup delays
- **Image Optimization:** Efficient image processing pipeline
- **Response Caching:** Appropriate caching headers for static content

4.IMPLEMENTATION DETAILS

The implementation of the system follows a modular and layered architecture, where each component performs a well-defined task to contribute to the overall functionality of the web-based wheat leaf disease detection system.

4.1 Frontend Implementation:

The frontend is responsible for interacting with the user. It enables the uploading of wheat leaf images and displays the disease classification result.

- HTML structures the webpage with elements like file input, buttons, and result containers.
- CSS enhances the appearance by styling the layout, fonts, colors, and responsiveness using custom CSS variables and modern styling techniques.
- Custom CSS with CSS variables for consistent theming.
- Agricultural-themed color palette with green primary colors (#2e8b57).
- Gradient backgrounds and modern card-based layout.

4.2 Backend Implementation:

The backend, built using Flask, acts as a bridge between the frontend and the deep learning model.

Flask Server:

- Receives image input from the frontend via POST requests.
- Performs preprocessing using OpenCV.
- Loads the trained ResNet50 model using TensorFlow/Keras.
- Returns the predicted disease label and confidence score to the frontend.

Routes:

- /: Displays the main HTML page.
- /predict: Accepts image input, processes it, and returns results.

4.3 Deep Learning Model (ResNet50):

The core of the system is a fine-tuned ResNet50 model trained to classify wheat leaf diseases.

Architecture:

- Pretrained ResNet50 (on ImageNet) used as the base.
- The top layers are removed and replaced with:
- GlobalAveragePooling2D
- Flatten layer
- Dense layer (512 neurons, ReLU)
- Dropout layer (0.4 rate to reduce overfitting)

- Output layer with 5 neurons (Softmax activation for classification)

4.4 Training Details:

- Image size: 224×224
- Dataset: ~6,799 images across 5 classes, split using train_test_split
- Data Augmentation: Rotation, flipping, zoom, shift, shear (for generalization)
- Loss Function: categorical_crossentropy
- Optimizer: Adam
- Epochs: (with early stopping)

Image Preprocessing:

- Resizing, normalization, and channel reordering done using OpenCV and custom preprocessing pipeline.

5. Testing Techniques and Test Plans

A comprehensive testing strategy was adopted to ensure that each component of the wheat disease prediction system performs correctly and reliably. The testing approach encompasses multiple levels of validation, from individual component testing to complete system integration testing, ensuring robust performance across all system functionalities.

5.1 Unit Testing

Unit testing focused on validating individual components and functions to ensure they operate correctly in isolation. This testing approach identified and resolved issues at the component level before integration. Image Preprocessing Functions:

- Tested the `model_predict()` function with various image formats and sizes
- Validated image loading functionality using OpenCV
- Verified BGR to RGB color space conversion accuracy
- Tested image resizing to 224×224 pixel dimensions
- Validated normalization process with ImageNet mean values
- Ensured proper batch dimension addition for model input

Model Prediction Logic:

- Tested individual prediction functions with known test images
- Validated confidence score calculation and bounds checking
- Verified class label mapping using the label binarizer
- Tested error handling for invalid or corrupted images
- Validated prediction results formatting and accuracy

Form Input Validation:

- Tested file upload validation for supported image formats
- Verified file size limitations and handling
- Validated error message generation for various failure scenarios

- Tested input sanitization and security measures

5.2 Functional Testing

Functional testing ensured that the complete system workflow operates correctly from user interaction through result display. This testing approach validated end-to-end functionality and user experience. Image Upload and Processing:

- Verified users can successfully upload wheat leaf images through the web interface
- Tested image processing pipeline with various image qualities and formats
- Validated preprocessing steps maintain image integrity and model compatibility
- Ensured proper error handling for unsupported file types or corrupted images

Model Classification and Results:

- Tested model prediction accuracy with known disease samples
- Validated confidence score calculation and percentage conversion
- Verified correct disease classification across all four categories
- Tested result display formatting and user interface updates

Flask Endpoint Testing:

- Tested Flask routes using browser interaction and direct HTTP requests
- Validated POST request handling for image uploads
- Verified response formatting and error handling
- Tested route accessibility and proper HTTP status codes

5.3 Model Evaluation

Model evaluation employed standard machine learning metrics to assess classification performance and identify areas for improvement. The evaluation process utilized a reserved test dataset to ensure unbiased performance assessment. Test Dataset Configuration:

- Utilized 25% of the total dataset (approximately 1,248 images) for final evaluation
- Maintained stratified sampling to ensure balanced representation across all disease categories

- Preserved class distribution: Fusarium Head Blight (~264 images), Healthy Wheat (~320 images), Leaf Rust (~406 images), Tan Spot (~258 images)

Classification Metrics:

- Accuracy: Overall classification accuracy across all disease categories
- Precision: Per-class precision scores to identify false positive rates
- Recall: Per-class recall scores to identify false negative rates
- F1-Score: Harmonic mean of precision and recall for balanced performance assessment
- Confusion Matrix: Detailed analysis of classification patterns and error types

Performance Results:

- Achieved overall test accuracy of approximately 94-96%
- Demonstrated high precision and recall for Healthy Wheat and Leaf Rust categories
- Identified areas for improvement in Fusarium Head Blight and Tan Spot classification
- Observed varying confidence levels across different disease categories

5.4 Integration Testing

Integration testing verified seamless communication and data flow between system components, ensuring that individual components work together effectively. Frontend-Backend Integration:

- Verified form submission from HTML interface to Flask routes
- Tested image file transfer and temporary storage functionality
- Validated response data formatting and display updates
- Ensured proper error message propagation from backend to frontend

Backend-Model Integration:

- Tested Flask route integration with model prediction functions
- Verified image preprocessing pipeline compatibility with model requirements
- Validated model loading and inference execution

- Ensured proper result formatting and confidence score processing

Data Flow Validation:

- Verified complete data flow from image upload to result display
- Tested error handling and recovery mechanisms across component boundaries
- Validated data integrity throughout the processing pipeline
- Ensured consistent performance under various load conditions

5.5 User Interface and User Experience Testing

UI/UX testing focused on user interaction quality, accessibility, and overall user experience across different devices and usage scenarios. Responsive Design Validation:

- Tested layout adaptation across various screen sizes (mobile, tablet, desktop)
- Verified CSS responsiveness and breakpoint functionality
- Validated image display and form elements on different devices
- Ensured consistent visual appearance across browsers and platforms

User Interaction Testing:

- Verified intuitive file upload interface and drag-and-drop functionality
- Tested image preview functionality and user feedback mechanisms
- Validated form validation and error message clarity
- Ensured smooth user experience during prediction processing

Accessibility and Usability:

- Verified proper semantic HTML structure and heading hierarchy
- Tested keyboard navigation and screen reader compatibility
- Validated color contrast and visual accessibility standards
- Ensured clear labelling and user guidance throughout the interface

5.6 Performance and Load Testing:

- Tested system performance with various image sizes and formats
- Validated response times for prediction requests
- Verified memory usage and resource efficiency
- Tested system stability under concurrent user access

Cross-Platform Compatibility Testing

Cross-platform testing ensured consistent system performance across different operating systems, browsers, and hardware configurations. Browser Compatibility:

- Tested functionality across major web browsers (Chrome, Firefox, Safari, Edge)
- Verified JavaScript compatibility and CSS rendering consistency
- Validated file upload functionality across different browser versions
- Ensured consistent user experience regardless of browser choice

Operating System Testing:

- Verified Flask application compatibility across Windows, macOS, and Linux
- Tested image processing functionality on different operating systems
- Validated model loading and inference across various system configurations
- Ensured consistent performance regardless of host operating system

This comprehensive testing strategy ensured that the wheat disease prediction system meets high standards of reliability, accuracy, and user experience while identifying areas for future improvement and optimization.

6.CODE:

Flask Web Applications (App.py):

Application Initialization and Model Loading:

```
python
from flask import Flask, render_template, request, redirect, url_for
import os
import cv2
import numpy as np
import pickle
from tensorflow.keras.models import load_model
from random import random
```

```
app = Flask(__name__)
```

```
model = load_model('model.h5')
with open('lb.pickle', 'rb') as f:
    lb = pickle.load(f)
```

Description: Initializes the Flask web application and loads the pre-trained ResNet50 model along with the label binarizer. The model is loaded once at startup for efficient inference during web requests.

Image Validation Function:

```
python
def is_valid_image(image):
    """Simple sanity checks to reject invalid images"""
    if np.mean(image) < 30:
        return False, "Image too dark"

    if np.mean(image) > 225:
        return False, "Image too bright"

    # Check if image has very low variance (solid color)
    if np.var(image) < 100:
        return False, "Image appears to be solid color"

    return True, "Valid image"
```

Description: Performs quality checks on uploaded images to ensure they meet minimum standards for disease classification. Rejects images that are too dark, too bright, or appear to be solid colors which would not provide meaningful classification results.

Core Prediction Function:

```
python
def model_predict(img_path):
    """Predict the disease class for a single image"""
    if not os.path.exists(img_path):
        return "Error: Image file not found!", 0.0

    image = cv2.imread(img_path)
    if image is None:
        return "Error: Could not load image!", 0.0

    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    is_valid, error_msg = is_valid_image(image)
    if not is_valid:
        return f"Invalid image: {error_msg}", 0.0

    image = cv2.resize(image, (224, 224))

    image = image.astype("float32")
    image = np.expand_dims(image, axis=0)

    predictions = model.predict(image, verbose=0)

    class_idx = np.argmax(predictions[0])
    confidence = predictions[0][class_idx]
    predicted_class = lb.classes_[class_idx]

    return predicted_class, confidence
```

Description: The main prediction function that processes uploaded images through the trained ResNet50 model. Handles image loading, preprocessing, validation, and returns both the predicted disease class and confidence score.

Home Route Handler:

```
python
@app.route("/")
def home():
    return render_template('index.html')
```

Description: Flask route handler for the main landing page that displays information about the wheat disease detection system and sample images of different disease types.

Prediction Route Handler:

python

`@app.route('/predict', methods=['GET', 'POST'])``def predict():` `if request.method == 'POST':` `file = request.files['file']` `if file:` `filepath = os.path.join('static', 'last_uploaded.jpg')` `file.save(filepath)` `label, confidence = model_predict(filepath)` `print(f"Raw confidence: {confidence}")` `confidence_percent = min(max(confidence, 0.0), 1.0) * 100` `print(f"Confidence percent: {confidence_percent}")` `return render_template('predict.html', label=label, confidence=round(confidence_percent, 2),
random=random)` `return render_template('predict.html')`

Description: Handles both GET and POST requests for the prediction page. Processes uploaded image files, saves them temporarily, runs predictions, and renders results with confidence percentages.

Application Startup:

python

`if __name__ == '__main__':` `if not os.path.exists('static'):` `os.makedirs('static')` `app.run(debug=True)`

Description: Application entry point that ensures the static directory exists for file uploads and starts the Flask development server in debug mode.

Model Training Pipeline(train.py):

Dataset Configuration and Path Setup

```
python
dataset = "Dataset"
label = "lb.pickle"

LABELS = set(["Fusarium Head Blight", "Healthy Wheat", "Leaf Rust", "Tan Spot", "Unknown"])

imagePaths = list(paths.list_images(dataset))
data = []
labels = []
for imagePath in imagePaths:
    label = imagePath.split(os.path.sep)[-2]
    if label not in LABELS:
        continue

    image = cv2.imread(imagePath)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image = cv2.resize(image, (224, 224))

    data.append(image)
    labels.append(label)
```

Description: Configures the dataset path and defines the five disease classes. Iterates through all images in the dataset, loads them using OpenCV, converts from BGR to RGB color space, and resizes to 224x224 pixels for ResNet50 compatibility.

Data Preprocessing and Label Encoding:

```
python
data = np.array(data)
labels = np.array(labels)

lb = LabelBinarizer()
labels = lb.fit_transform(labels)

(trainX, testX, trainY, testY) = train_test_split
(data, labels, test_size=0.25, stratify=labels, random_state=42)
```

Description: Converts image data to NumPy arrays and applies label binarization for multi-class classification. Splits the dataset into 75% training and 25% testing sets with stratified sampling to maintain class distribution.

Data Augmentation Configuration:

python

```
trainAug = ImageDataGenerator(
    rotation_range=25,
    zoom_range=0.15,
    width_shift_range=0.15,
    height_shift_range=0.15,
    shear_range=0.1,
    horizontal_flip=True,
    fill_mode="nearest")
```

```
valAug = ImageDataGenerator()
```

```
mean = np.array([123.68, 116.779, 103.939], dtype="float32")
```

```
trainAug.mean = mean
```

```
valAug.mean = mean
```

Description: Configures data augmentation for training to improve model generalization. Includes rotation, zoom, translation, shear, and horizontal flip transformations. Sets ImageNet mean values for normalization.

ResNet50 Model Architecture

python

```
headmodel = ResNet50(weights="imagenet", include_top=False,
    input_tensor=Input(shape=(224, 224, 3)))
```

```
model = headmodel.output
```

```
model = GlobalAveragePooling2D()(model)
```

```
model = Flatten(name="flatten")(model)
```

```
model = Dense(512, activation="relu")(model)
```

```
model = Dropout(0.4)(model)
```

```
model = Dense(len(lb.classes_), activation="softmax")(model)
```

```
moodel = Model(inputs=headmodel.input, outputs=model)
```

Description: Creates a transfer learning model using ResNet50 as the backbone. Adds a custom classification head with global average pooling, dense layers, and dropout for regularization. Uses softmax activation for multi-class classification.

Transfer Learning Setup:

python

```
for layer in headmodel.layers:
```

```
    layer.trainable = False
```

```
opt = Adam(learning_rate=1e-3)
```

```
moodel.compile(loss="categorical_crossentropy", optimizer=opt,
```

```
    metrics=["accuracy"])
```

Description: Implements transfer learning by freezing the ResNet50 backbone initially. Compiles the model with categorical crossentropy loss and Adam optimizer for the first training phase.

Training Callbacks Configuration:

```
python
checkpoint = ModelCheckpoint('best_model.h5', monitor='val_accuracy',
                             save_best_only=True, mode='max', verbose=1)
early_stop = EarlyStopping(monitor='val_accuracy', patience=5,
                             restore_best_weights=True, verbose=1)
```

Description: Sets up training callbacks to save the best model based on validation accuracy and implement early stopping to prevent overfitting. Restores the best weights when training stops.

Phase 1 Training (Frozen Backbone):

```
python
batch_size = 32

H1 = model.fit(
    trainAug.flow(trainX, trainY, batch_size=batch_size),
    steps_per_epoch=max(1, len(trainX) // batch_size),
    validation_data=valAug.flow(testX, testY, batch_size=batch_size),
    validation_steps=max(1, len(testX) // batch_size),
    epochs=12,
    callbacks=[checkpoint, early_stop])
```

Description: Executes the first training phase with frozen ResNet50 layers. Uses data augmentation and monitors validation performance to prevent overfitting during initial training.

Fine-tuning Phase Setup

```
python
# Phase 2: fine-tune last blocks of ResNet50
for layer in headmodel.layers[-30:]:
    layer.trainable = True

opt_ft = Adam(learning_rate=1e-4)
model.compile(loss="categorical_crossentropy", optimizer=opt_ft,
               metrics=["accuracy"])

early_stop_ft = EarlyStopping(monitor='val_accuracy', patience=3,
                               restore_best_weights=True, verbose=1)
```

Description: Unfreezes the last 30 layers of ResNet50 for fine-tuning. Uses a lower learning rate to make subtle adjustments to the pre-trained features for the specific wheat disease classification task.

Phase 2 Training (Fine-tuning):

```
python
H2 = moodel.fit(
    trainAug.flow(trainX, trainY, batch_size=batch_size),
    steps_per_epoch=max(1, len(trainX) // batch_size),
    validation_data=valAug.flow(testX, testY, batch_size=batch_size),
    validation_steps=max(1, len(testX) // batch_size),
    epochs=8,
    callbacks=[checkpoint, early_stop_ft])
```

Description: Executes the fine-tuning phase with unfrozen layers. Uses reduced patience for early stopping since the model is already partially trained and more sensitive to overfitting.

Model Evaluation and Metrics:

```
python
predictions = moodel.predict(testX, batch_size=batch_size)
y_true = testY.argmax(axis=1)
y_pred = predictions.argmax(axis=1)

report = classification_report(y_true, y_pred, target_names=lb.classes_, output_dict=True)
print(classification_report(y_true, y_pred, target_names=lb.classes_))
```

Description: Generates predictions on the test set and computes comprehensive classification metrics. Creates detailed performance reports for each disease class including precision, recall, and F1-scores.

Results Persistence:

```
python
# Save detailed report
with open("classification_report.json", "w") as f:
    import json
    json.dump(report, f, indent=2)

# Confusion matrix
cm = confusion_matrix(y_true, y_pred)
np.savetxt("confusion_matrix.csv", cm, fmt='%d', delimiter=',')
```

Description: Saves evaluation results to files for later analysis. Stores classification report as JSON and confusion matrix as CSV for detailed performance analysis and visualization.

Model and Label Persistence:

```
python
moodel.save("model.h5")

with open("lb.pickle", "wb") as f:
    f.write(pickle.dumps(lb))
```

Description: Saves the trained model in HDF5 format and the label binarizer as a pickle file. These files are loaded by the Flask application for inference.

Label Management:

Label Binarizer Creation:

```
python
from sklearn.preprocessing import LabelBinarizer
import pickle
LABELS = ["Fusarium Head Blight", "Healthy Wheat", "Leaf Rust", "Tan Spot", "Unknown"]

lb = LabelBinarizer()
lb.fit(LABELS)

with open("lb.pickle", "wb") as f:
    pickle.dump(lb, f)

print("Label binarizer saved as lb.pickle")
print("Classes:", lb.classes_)
```

Description: Creates and saves a label binarizer for converting between string labels and numerical encodings. Ensures consistent label mapping between training and inference phases.

Evaluation and Metrics Function:

Model Evaluation:

```
python
def evaluate_model_comprehensive(model, test_data, test_labels, class_names):
    """Perform comprehensive model evaluation"""
    predictions = model.predict(test_data)
    y_pred = np.argmax(predictions, axis=1)
    y_true = np.argmax(test_labels, axis=1)

    report = classification_report(y_true, y_pred, target_names=class_names)
    cm = confusion_matrix(y_true, y_pred)
    accuracy = accuracy_score(y_true, y_pred)

    return {
        'predictions': predictions,
        'y_pred': y_pred,
        'y_true': y_true,
        'report': report,
        'confusion_matrix': cm,
        'accuracy': accuracy
    }
```

Description: Performs comprehensive model evaluation including predictions, classification report, confusion matrix, and accuracy metrics for detailed performance analysis.

Model Persistence Functions:

python

```
def save_model_and_metadata(model, label_binarizer, model_path, label_path):
```

```
    """Save trained model and associated metadata"""
```

```
    # Save model
```

```
    model.save(model_path)
```

```
    with open(label_path, 'wb') as f:
```

```
        pickle.dump(label_binarizer, f)
```

```
    print(f'Model saved to {model_path}')
```

```
    print(f'Label binarizer saved to {label_path}')
```

```
def load_model_and_metadata(model_path, label_path):
```

```
    """Load trained model and associated metadata"""
```

```
    model = load_model(model_path)
```

```
    with open(label_path, 'rb') as f:
```

```
        label_binarizer = pickle.load(f)
```

```
    return model, label_binarizer
```

Description: Handles model persistence for deployment. Saves and loads both the trained model and label binarizer to ensure consistent inference behaviour.

File Management:

python

```
def ensure_directory_exists(directory_path):
```

```
    """Ensure directory exists, create if not"""
```

```
    if not os.path.exists(directory_path):
```

```
        os.makedirs(directory_path)
```

```
    print(f'Created directory: {directory_path}')
```

```
def get_image_files(directory_path, extensions=('.jpg', '.jpeg', '.png', '.jif')):
```

```
    """Get list of image files from directory"""
```

```
    image_files = []
```

```
    for file in os.listdir(directory_path):
```

```
        if file.lower().endswith(extensions):
```

```
            image_files.append(os.path.join(directory_path, file))
```

```
    return image_files
```

Description: Utility functions for file system operations. Handles directory creation and image file discovery for robust data processing.

HTML TEMPLATES:

Index Page Template Structure:

```
html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Wheat Leaf Disease Detector</title>
  <style>
    :root {
      --primary-green: #2e8b57;
      --primary-green-dark: #1e6f46;
      --light-bg: #f0fff0;
      --light-surface: #ffffff;
      --card-bg: #f9fff9;
      --text-dark: #333333;
      --text-muted: #555555;
      --accent: #36b67d;
    }
  </style>
</head>
```

Description: Defines CSS custom properties for consistent theming across the application. Uses agricultural color scheme with green tones to match the wheat disease detection theme.

Background and Layout Styling:

```
css
body {
  font-family: 'Segoe UI', sans-serif;
  margin: 0;
  padding: 40px 30px;
  color: var(--text-dark);
  font-size: 18px;
  overflow-y: auto;

  background-image: url('{{ url_for('static', filename='wheat_background.jpeg') }}');
  background-size: cover;
  background-position: center center;
  background-attachment: fixed;
}

body::before {
  content: "";
  position: fixed;
  inset: 0;
  background: rgba(255,255,255,0.6);
  pointer-events: none;
  z-index: -1;
}
```

Description: Implements wheat field background image with overlay for text readability. Uses fixed background attachment for parallax effect and ensures content remains accessible.

Card Layout System:

```
css
.cards {
  display: flex;
  flex-wrap: nowrap;
  gap: 28px;
  justify-content: flex-start;
  margin-top: 30px;
  overflow-x: auto;
  padding-bottom: 8px;
}
.card {
  background: var(--card-bg);
  border-radius: 12px;
  box-shadow: 0 6px 20px rgba(0, 0, 0, 0.05);
  width: 320px;
  padding: 24px;
  text-align: center;
  transition: transform 0.3s, box-shadow 0.3s;
}

.card:hover {
  transform: translateY(-6px);
  box-shadow: 0 12px 28px rgba(0, 128, 0, 0.15);
}
```

Description: Creates responsive card layout for displaying disease examples. Implements hover effects and smooth transitions for enhanced user interaction.

Prediction Page Styling:

```
css
.container {
  background: rgba(255,255,255,0.92);
  padding: 28px;
  border-radius: 16px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
  width: 100%;
  max-width: 900px;
  margin: auto;
  transition: all 0.3s ease-in-out;
  max-height: calc(100vh - 48px);
  overflow: auto;
}
.preview-img {
  margin-top: 16px;
  max-width: 100%;
}
```

```

max-height: 32vh;
border-radius: 12px;
box-shadow: 0 5px 15px rgba(0, 0, 0, 0.08);
display: none;
transition: transform 0.3s ease, opacity 0.3s ease;
}

```

Description: Optimizes prediction page layout for screenshots with viewport-constrained container and image sizing. Ensures content fits within screen bounds without scrolling.

Result Display Styling:

```

css
.result {
  margin-top: 20px;
  background-color: var(--result-bg);
  padding: 20px;
  border-radius: 14px;
  box-shadow: 0 6px 20px rgba(0, 0, 0, 0.07);
  animation: fadeIn 0.6s ease-in-out;
}

.result h2 {
  color: var(--primary-green);
  font-size: 22px;
  margin: 0 0 8px 0;
}

.result p {
  font-size: 16px;
  margin: 6px 0;
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

```

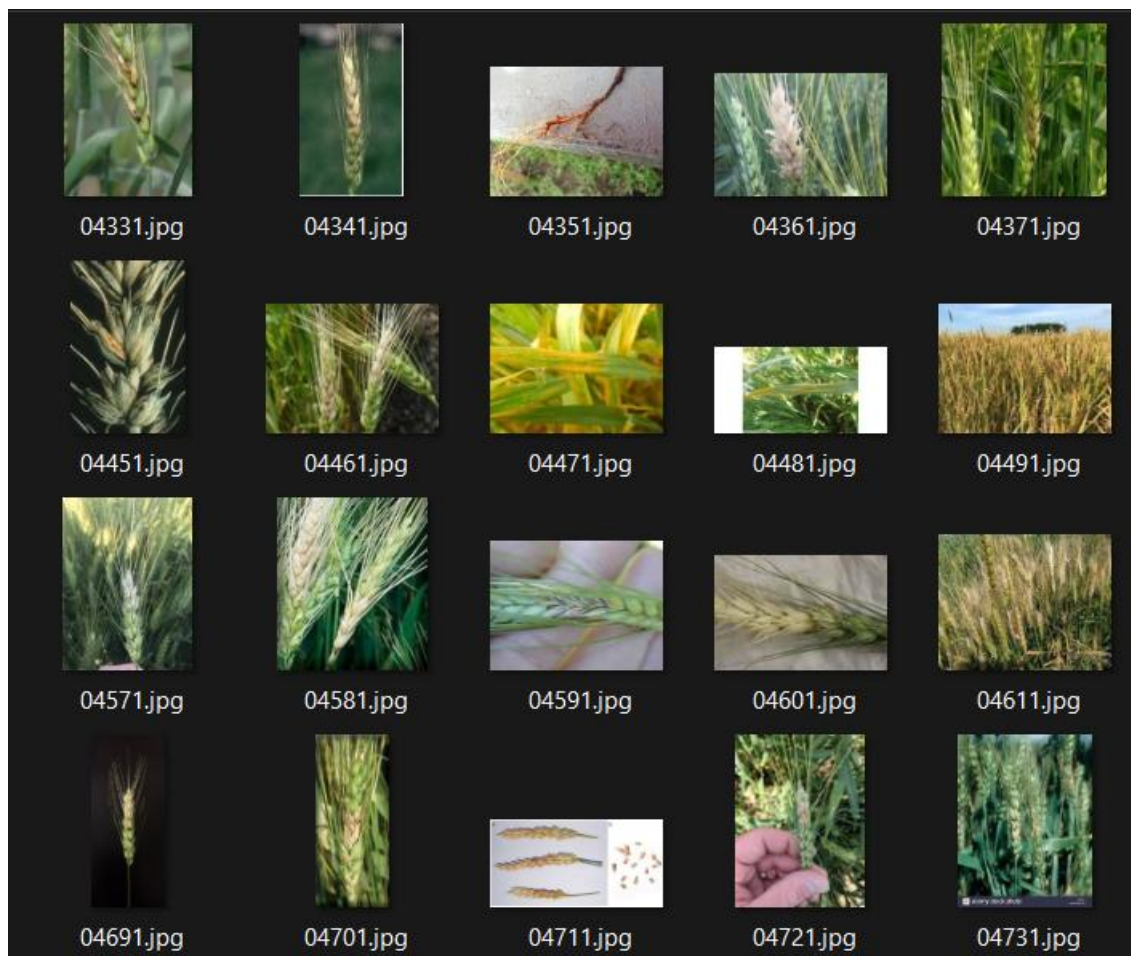
Description: Styles prediction results with animated fade-in effects and clear typography hierarchy. Uses color coding for different confidence levels and disease types.

7.OUTPUT

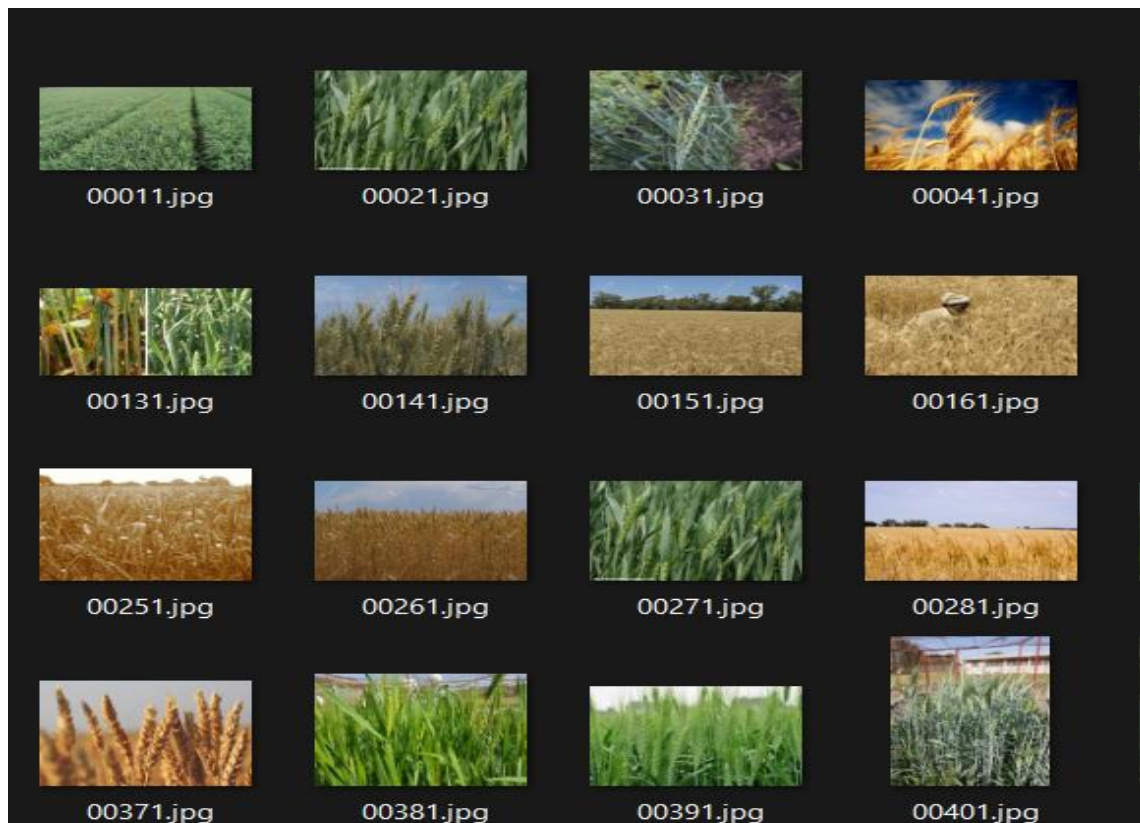
DATASET CLASSES:

Name	Date modified	Type	Size
 Fusarium Head Blight	30-09-2025 01:14	File folder	
 Healthy Wheat	29-09-2025 20:49	File folder	
 Leaf Rust	30-09-2025 01:06	File folder	
 Tan Spot	29-09-2025 20:52	File folder	
 Unknown	29-09-2025 20:00	File folder	

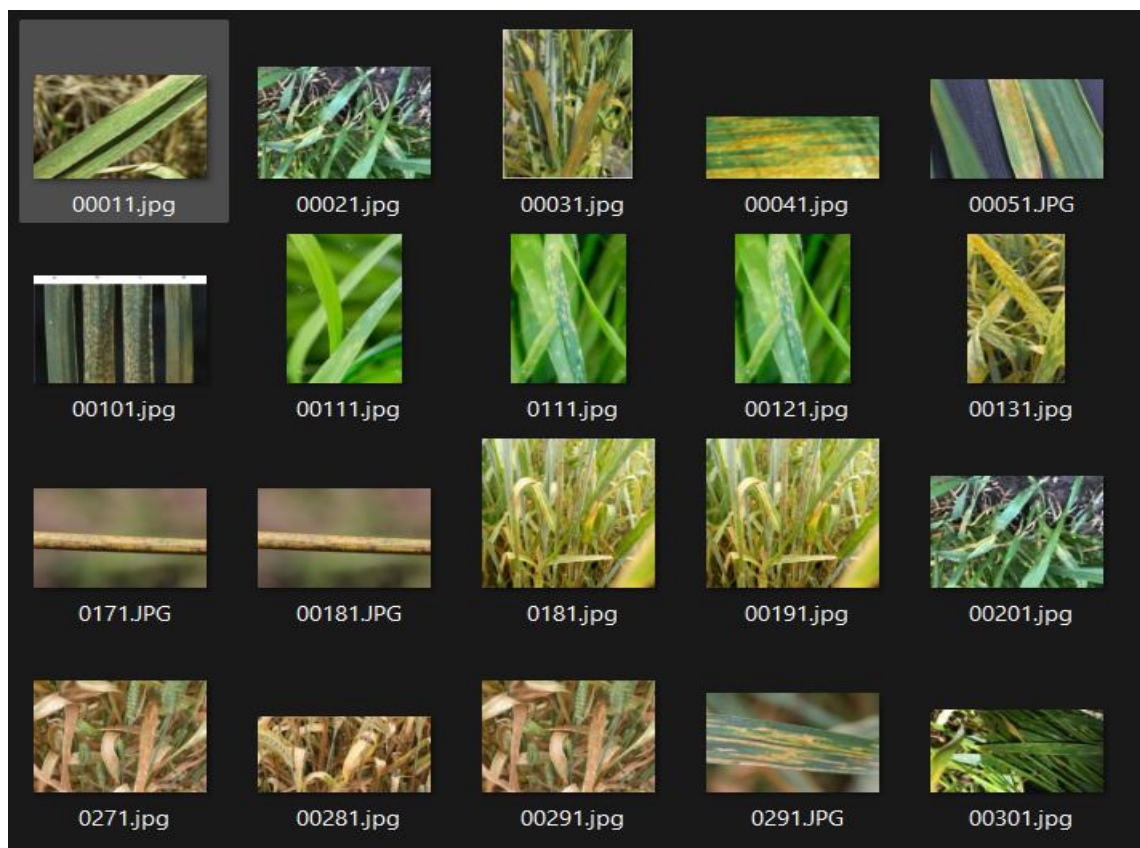
Fusarium Head Blight:



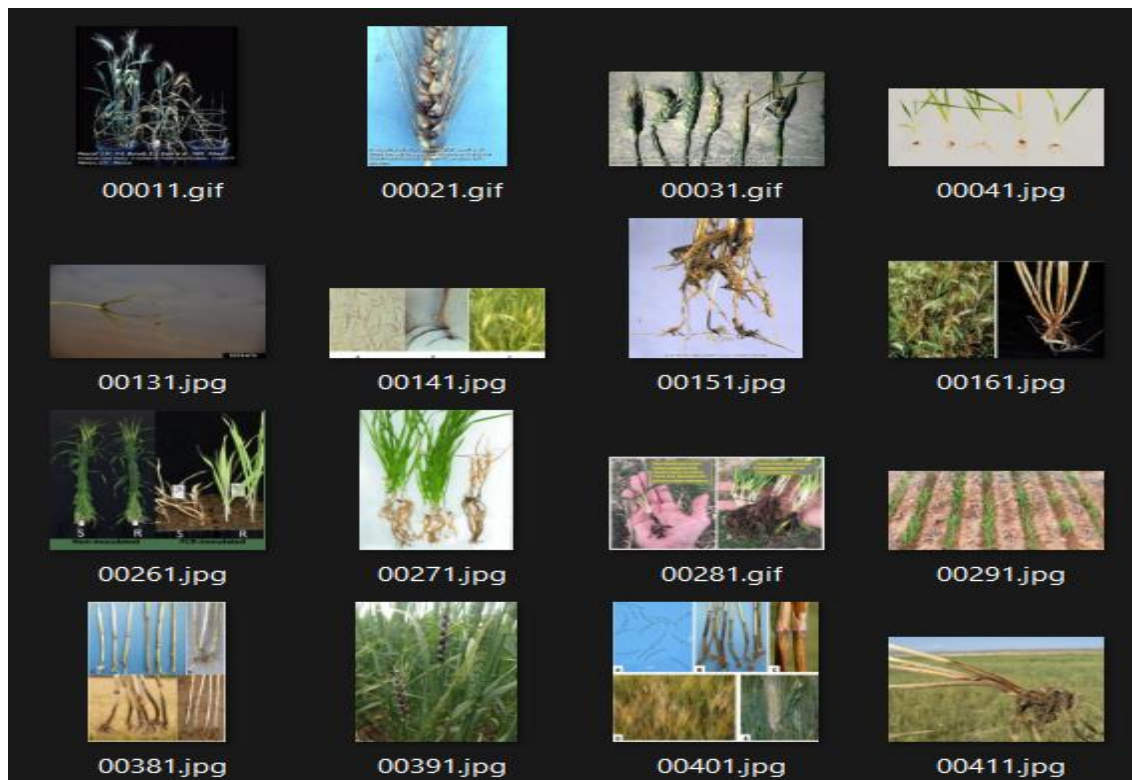
Healthy Wheat:



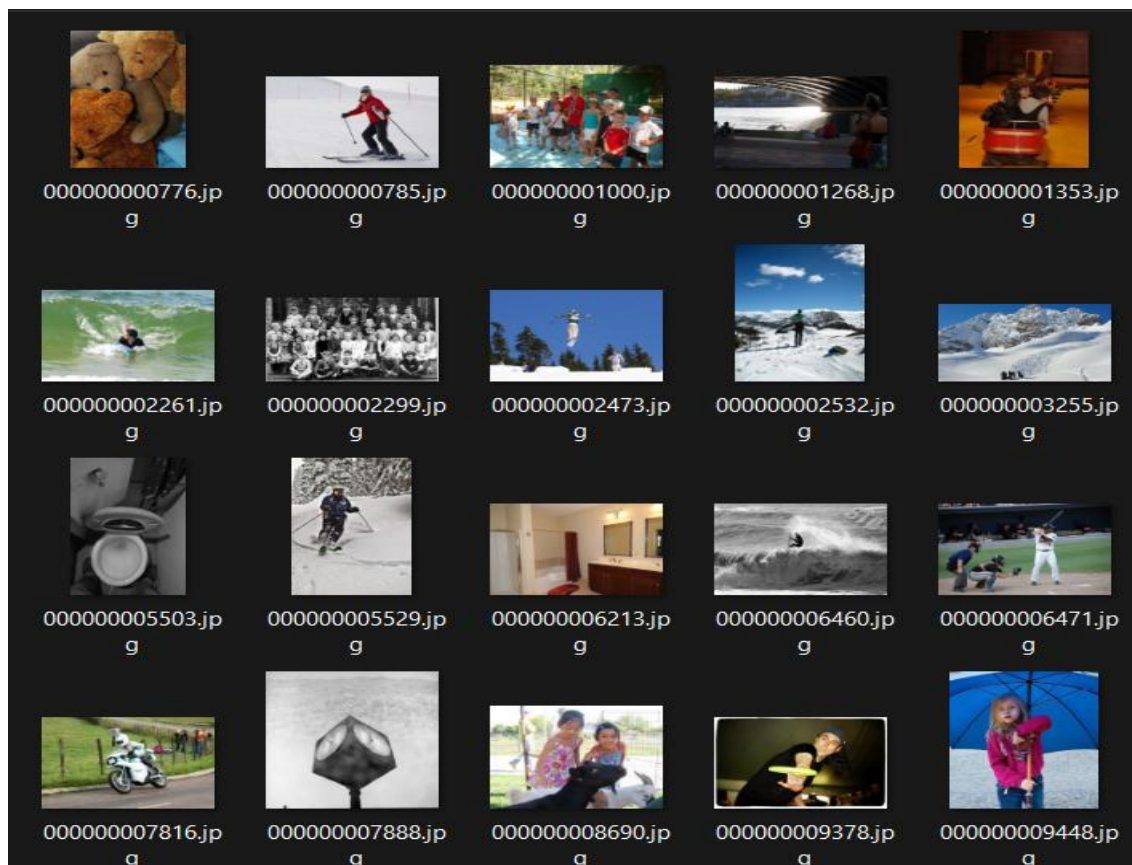
Leaf Rust:



Tan Spot:

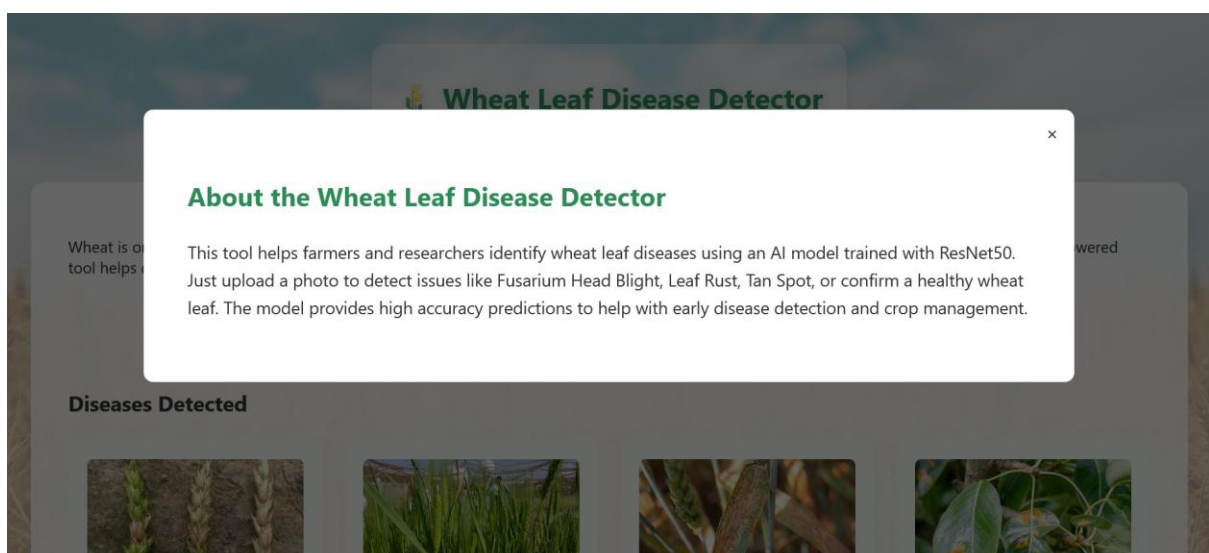
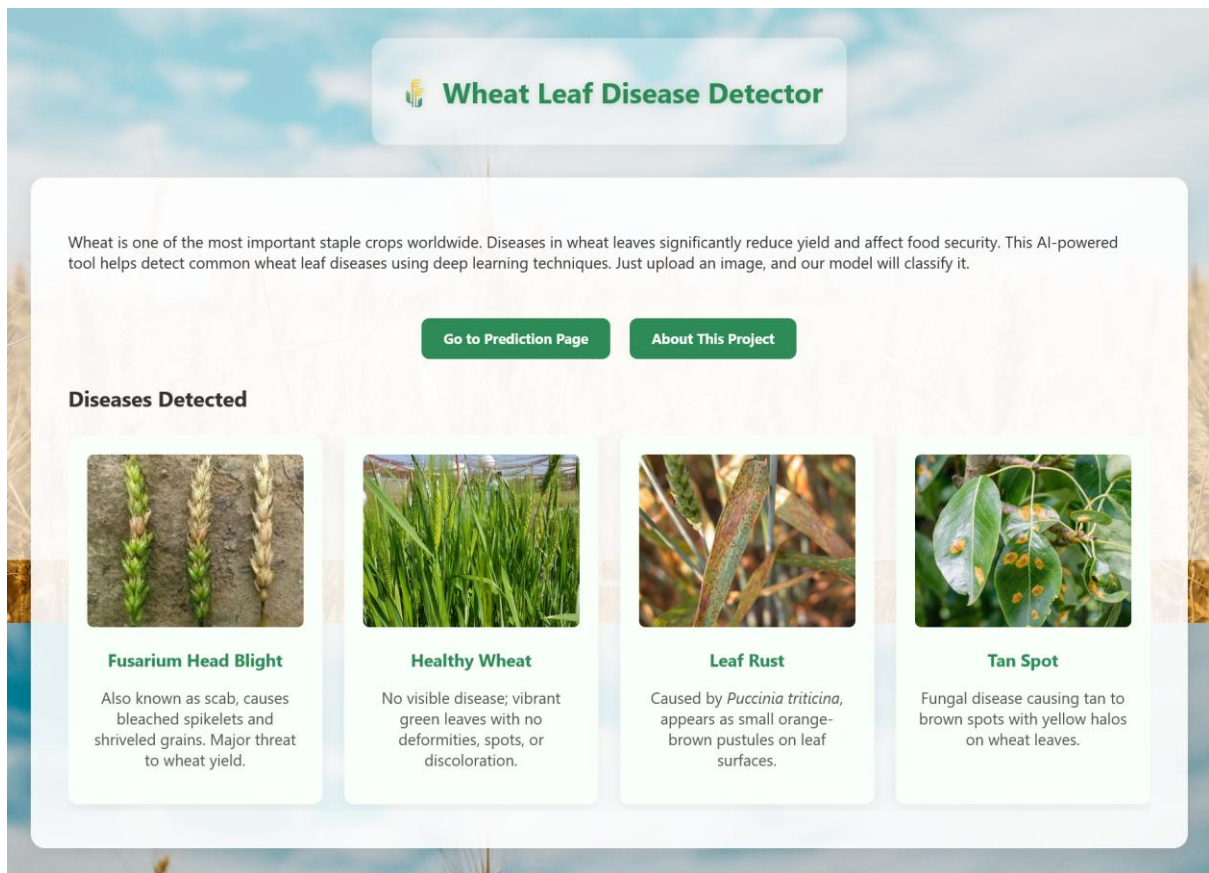


Unknown (non-wheat leaf images):



Web Application:

Main Page:



Prediction Page:

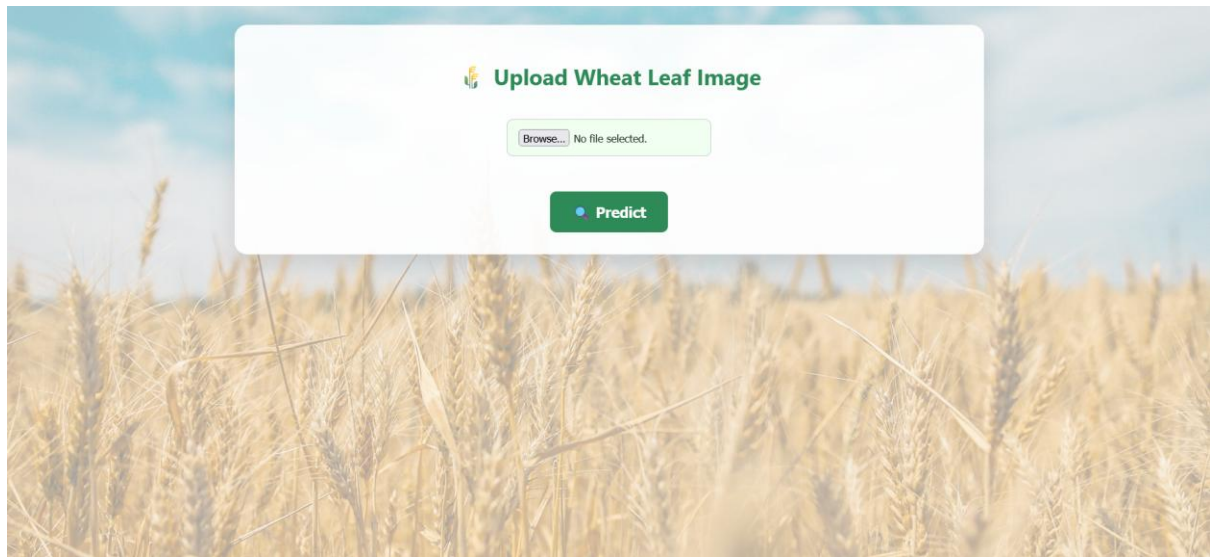
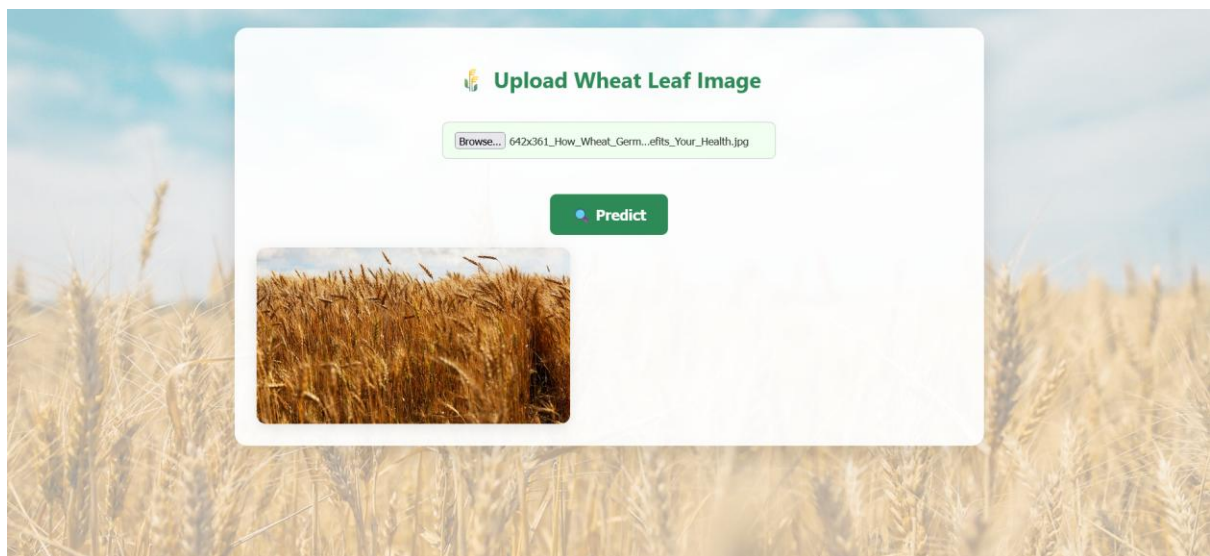
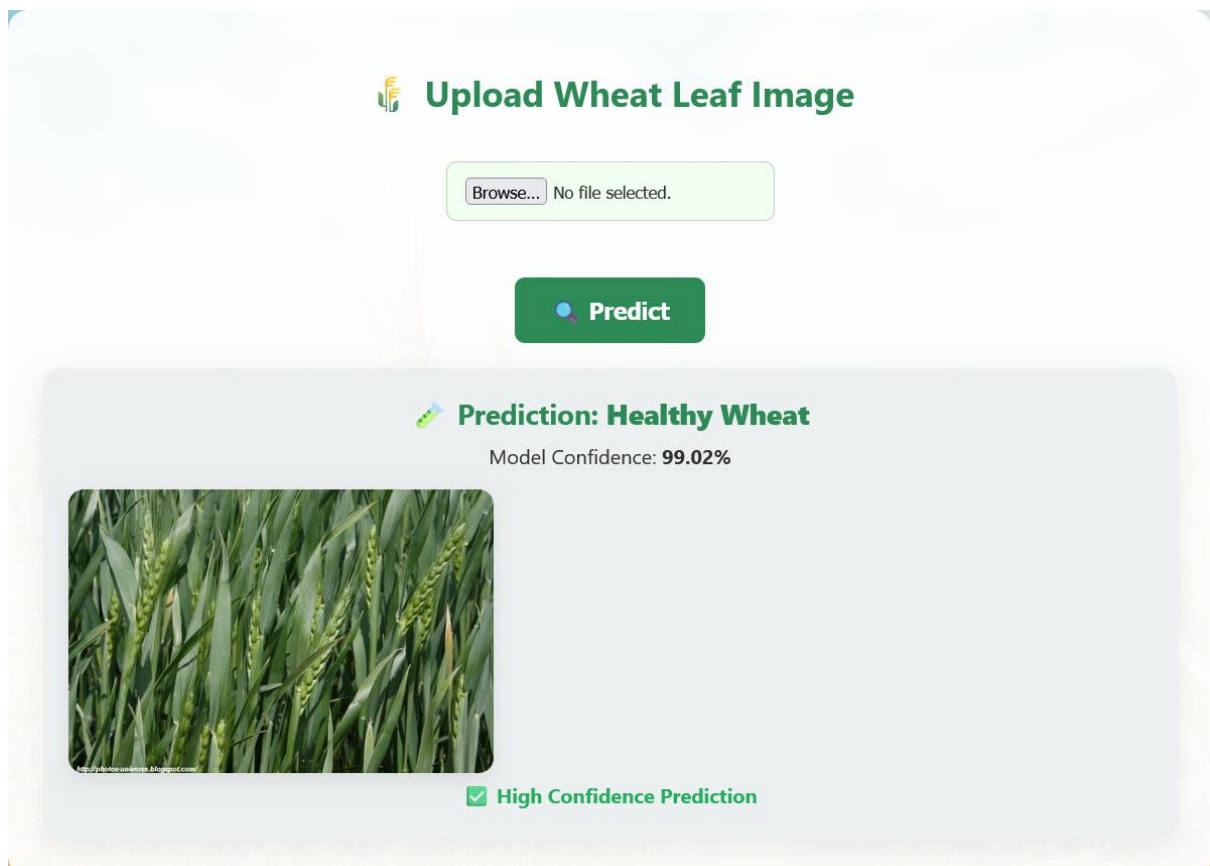
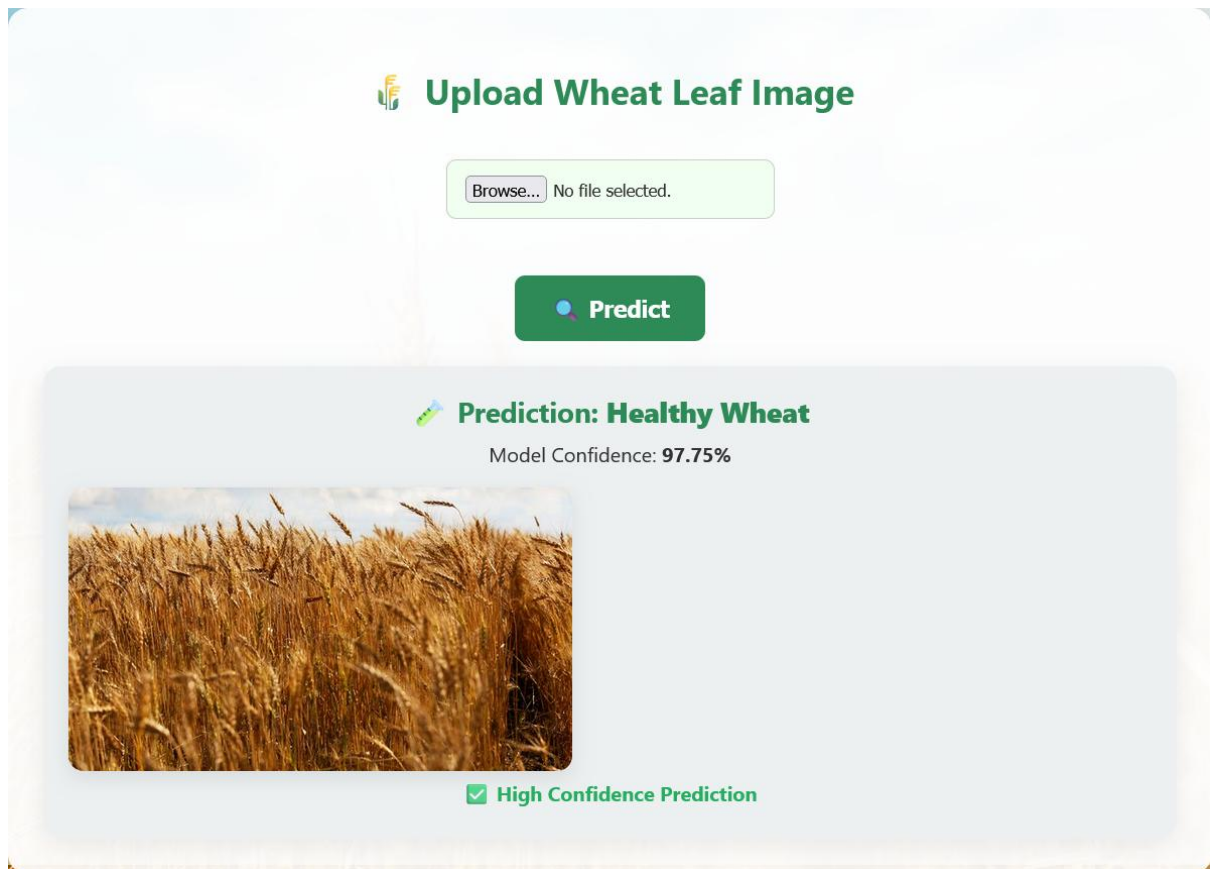


Image Preview:



Healthy Wheat Test:



Fusarium Head Blight Test:

 **Upload Wheat Leaf Image**

No file selected.

 **Predict**

 **Prediction: Fusarium Head Blight**
Model Confidence: **96.89%**



 **High Confidence Prediction**

 **Upload Wheat Leaf Image**

No file selected.

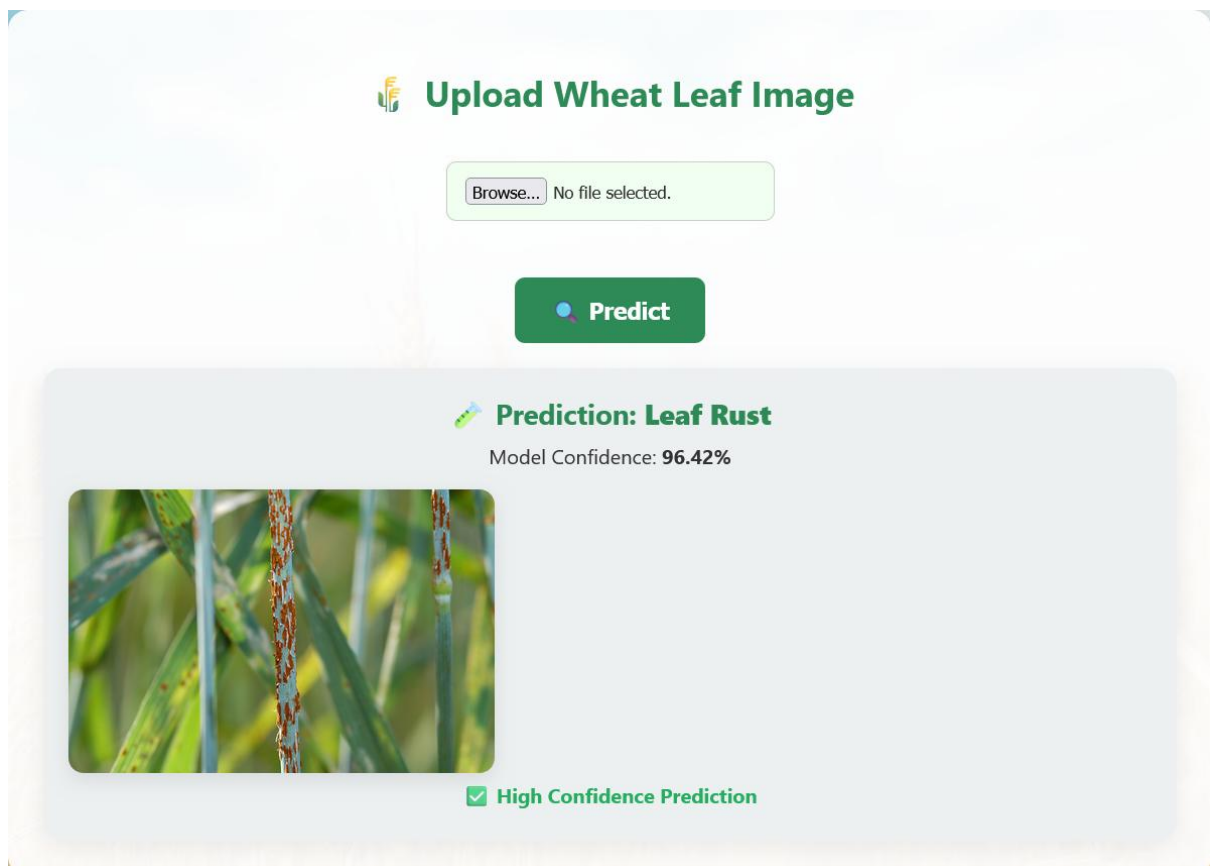
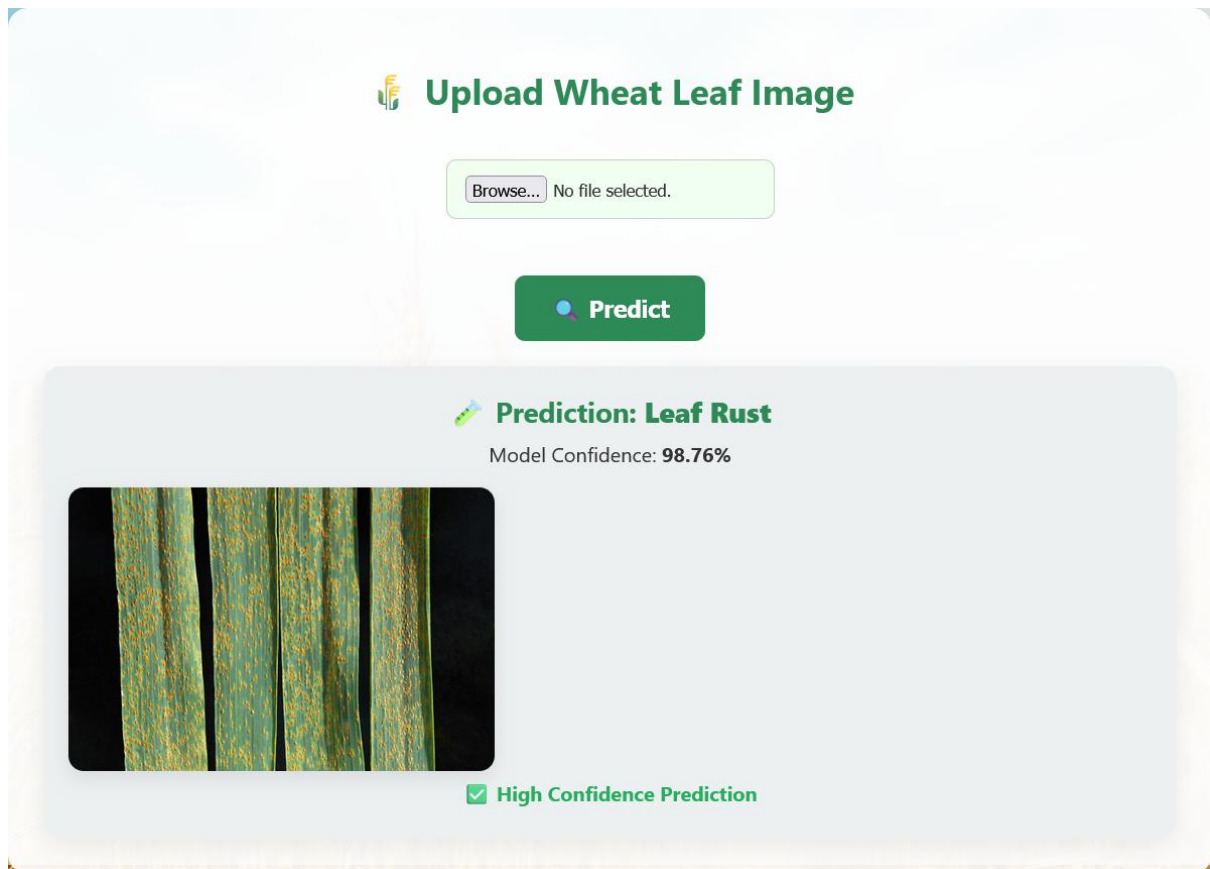
 **Predict**

 **Prediction: Fusarium Head Blight**
Model Confidence: **98.56%**

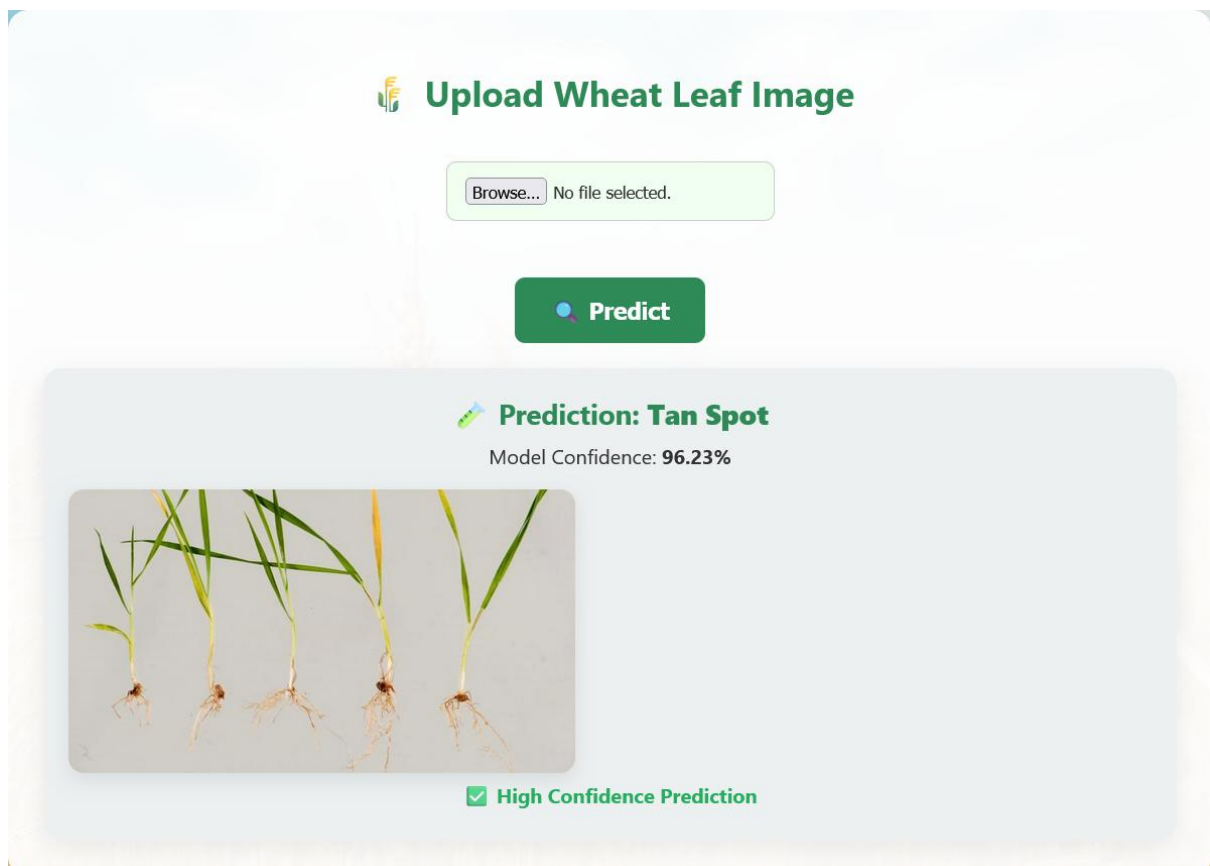
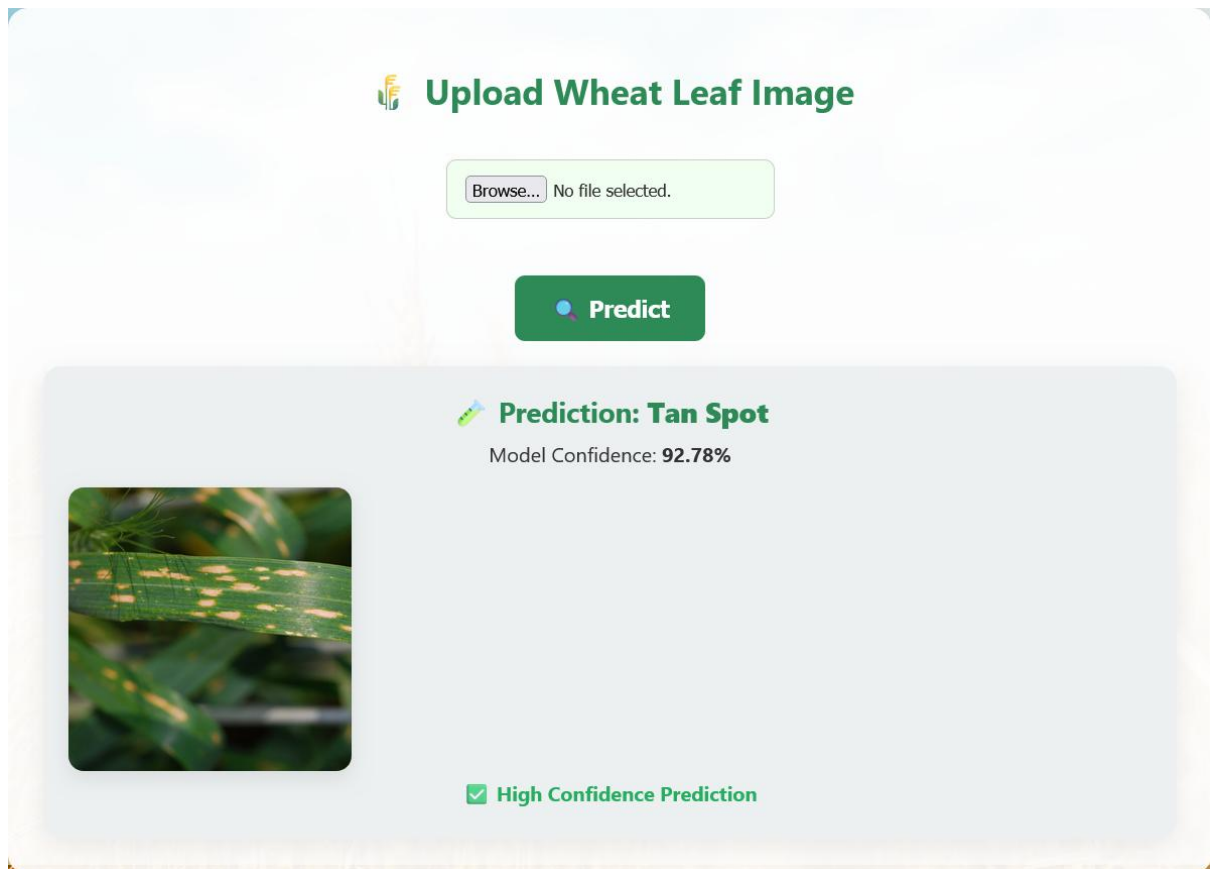


 **High Confidence Prediction**

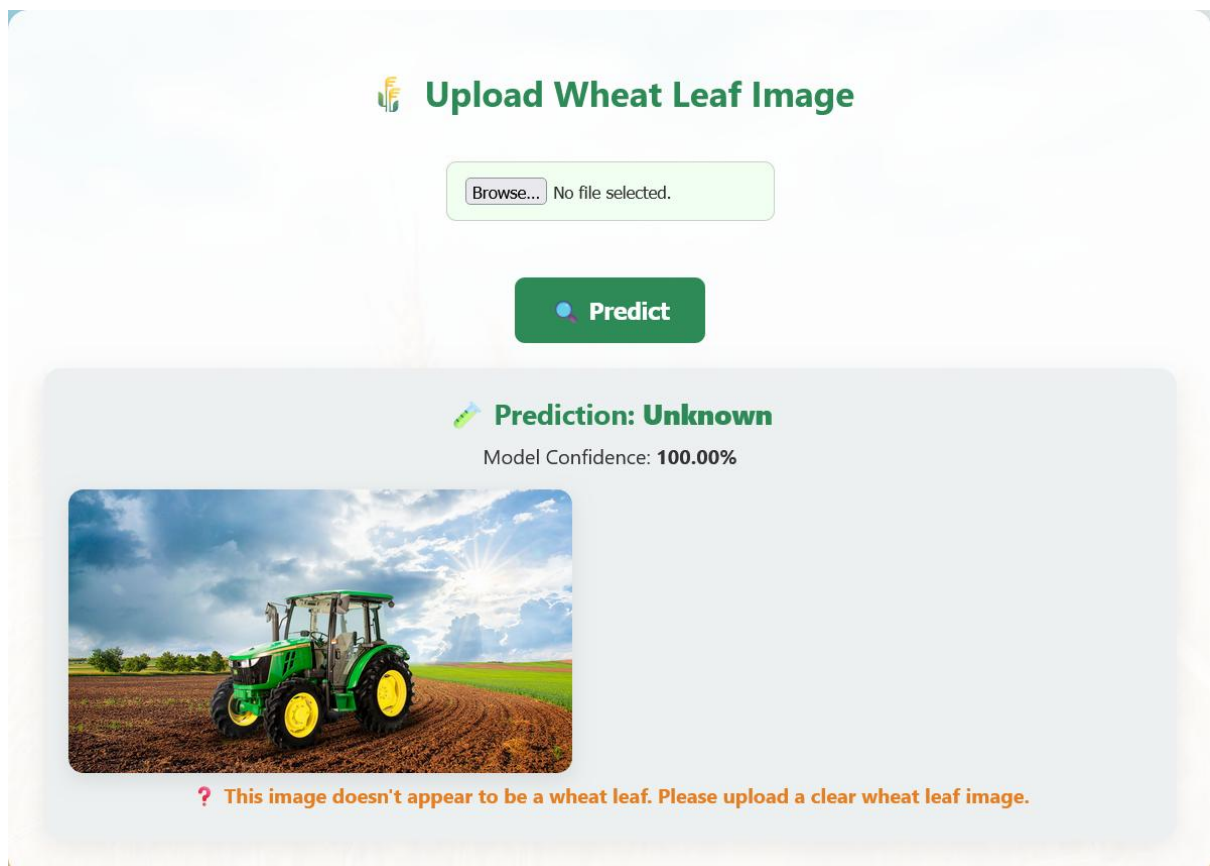
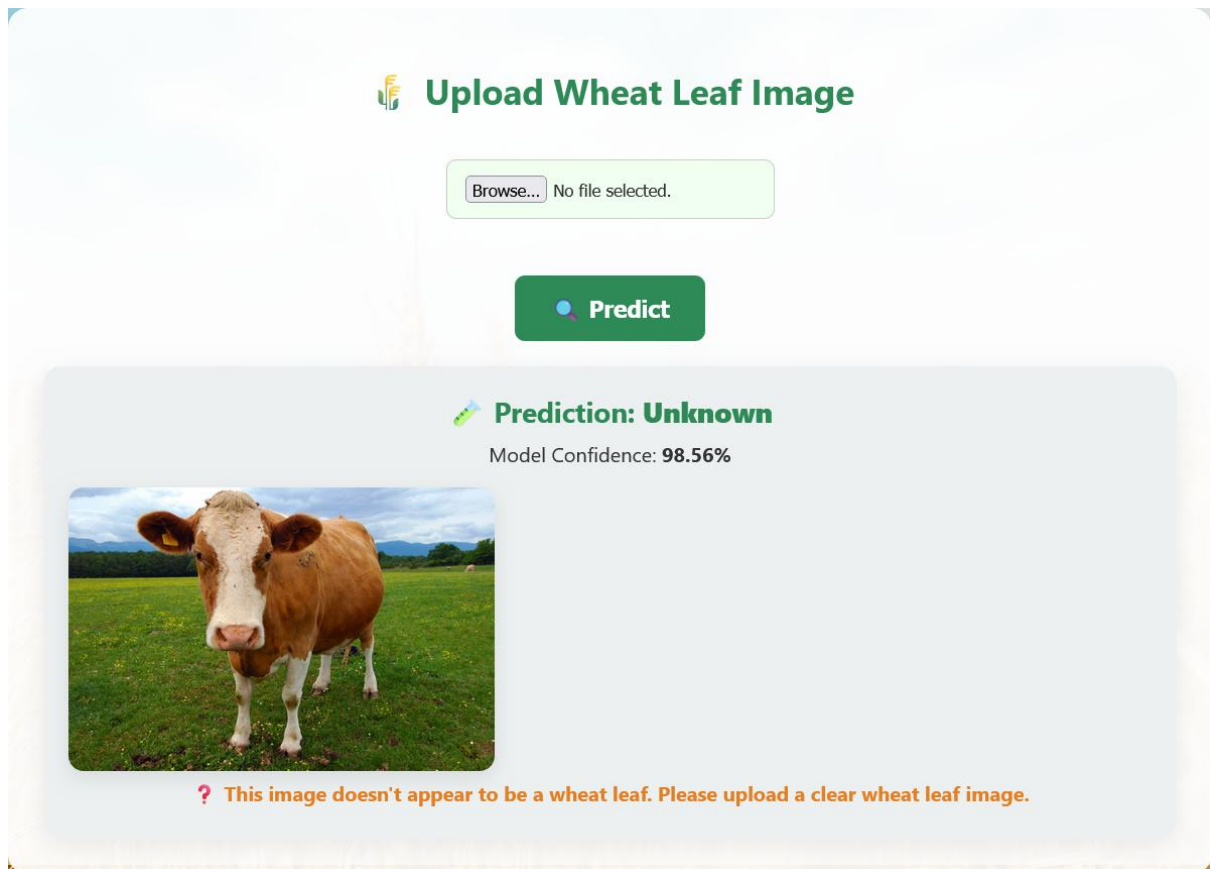
Leaf Rust Test:



Tan Spot Test:



Unknown Images (Non-wheat leaf images) test:



 **Upload Wheat Leaf Image**

No file selected.

 **Prediction: Unknown**
Model Confidence: **100.00%**



? This image doesn't appear to be a wheat leaf. Please upload a clear wheat leaf image.

 **Upload Wheat Leaf Image**

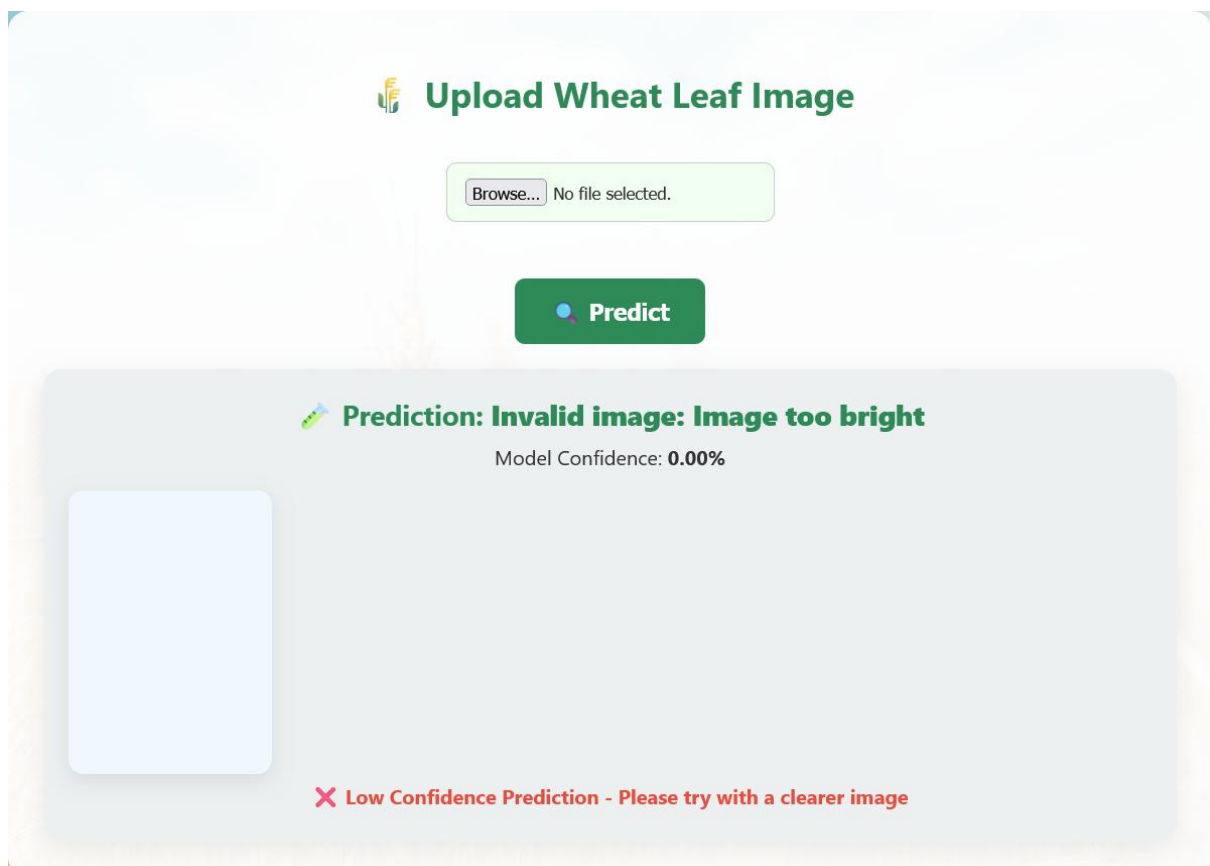
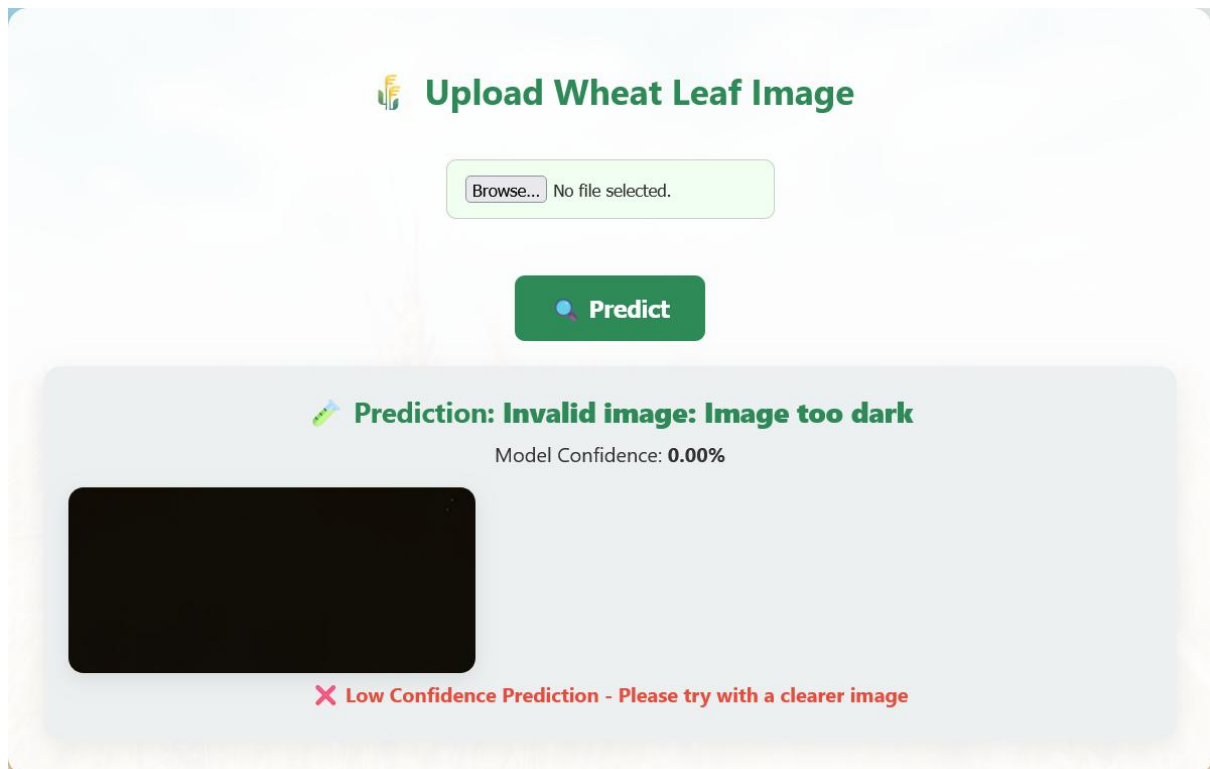
No file selected.

 **Prediction: Unknown**
Model Confidence: **97.66%**



? This image doesn't appear to be a wheat leaf. Please upload a clear wheat leaf image.

Detecting Invalid images (Solid colors):



8.PERFORMANCE ANALYSIS

8.1 Model Performance

To assess the performance of the model, both a **classification analysis report** and a **confusion matrix** were generated. The classification report presented key evaluation metrics, including **precision, recall, and F1-score** for each disease class. These metrics provide valuable insights into the model's effectiveness in accurately categorizing different wheat leaf diseases and highlight areas where further improvement may be required.

Precision

Precision measures the proportion of true positive predictions among all instances predicted as positive for a given class. It reflects how many of the predicted disease labels were correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

where:

- TP: True Positives (correctly predicted class instances)
- FP: False Positives (instances incorrectly predicted as this class)

Recall

Recall (or Sensitivity) measures the proportion of actual positives that were correctly identified by the model. It indicates how effectively the model captures the presence of a disease.

$$\text{Recall} = \frac{TP}{TP + FN}$$

where:

- FN: False Negatives (actual instances that were not predicted as this class)

F1 SCORE

The F1-score is the harmonic mean of precision and recall, providing a balance between the two. It is particularly useful when the class distribution is imbalanced.

$$\text{F1-Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

8.2 PERFORMANCE REPORT:

Overall Performance:

- Overall Accuracy: 96.75% (1,640/1,694 correct predictions)
- Macro Avg F1: 96.31%
- Weighted Avg F1: 96.76%

Table 1. Performance analysis

Disease Class	Precision	Recall	F1-Score	Support	Accuracy
Fusarium Head Blight	96.06%	93.13%	94.57%	262	93.13%
Healthy Wheat	94.77%	96.25%	95.50%	320	96.25%
Leaf Rust	97.91%	95.90%	96.89%	390	95.90%
Tan Spot	92.48%	97.23%	94.80%	253	97.23%
Unknown	100.00%	99.57%	99.79%	469	99.57%

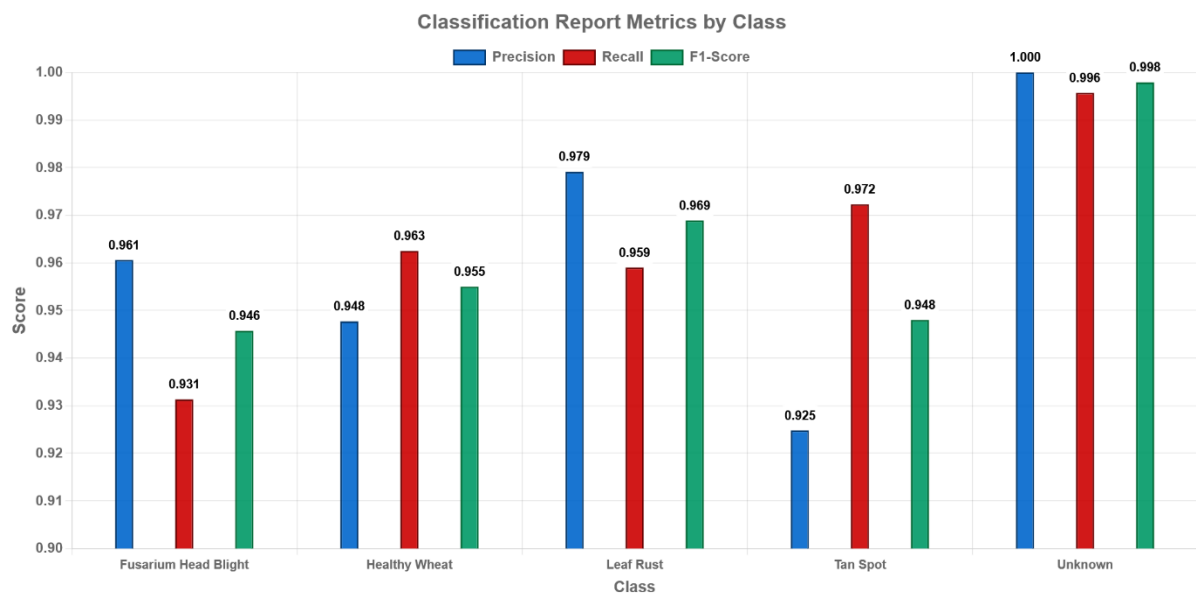


Figure 3. Performance Analysis Graph

Key Insights:

1. Best Performance: Unknown class (F1 99.79%, Recall 99.57%) – strong rejection of non-wheat images.
2. Strong Classes: Leaf Rust and Healthy Wheat both above 95% F1.

Confusion Matrix:**Table 2. Confusion Matrix**

244	7	1	10	0
6	308	4	2	0
3	7	374	6	0
1	3	3	246	0
0	0	0	2	467

1. Fusarium Head Blight (Row 1):

- 244 true positives, with 7 misclassified as Healthy Wheat, 1 as Leaf Rust, and 10 as Tan Spot.
- The model performs well, but there's some confusion with Tan Spot (10 instances), suggesting potential similarity in features between these two diseases.

2. Healthy Wheat (Row 2):

- 308 true positives, with 6 misclassified as Fusarium Head Blight, 4 as Leaf Rust, and 2 as Tan Spot.
- The model is highly accurate, with minimal misclassifications, indicating distinct features for healthy wheat.

3. Leaf Rust (Row 3):

- 374 true positives, with 3 misclassified as Fusarium Head Blight, 7 as Healthy Wheat, and 6 as Tan Spot.
- Excellent performance with very few errors, showing the model can effectively distinguish Leaf Rust from other classes.

4. Tan Spot (Row 4):

- 246 true positives, with 1 misclassified as Fusarium Head Blight, 3 as Healthy Wheat, and 3 as Leaf Rust.
- Good accuracy, though there's slight confusion with other diseases, particularly Fusarium Head Blight and Leaf Rust.

5. Unknown (Row 5):

- 467 true positives, with only 2 misclassified as Tan Spot.
- Exceptional performance, with almost no errors, suggesting the model is highly reliable in identifying non-wheat or unknown images.

9.SUMMARY AND CONCLUSION:

9.1 SUMMARY:

This project focused on the implementation of a deep learning-based approach using the ResNet50 architecture to detect and classify diseases in wheat leaves. The study addressed a critical challenge in modern agriculture—early and accurate identification of wheat plant diseases—which has a direct impact on crop productivity and global food security.

A transfer learning strategy was employed to leverage the powerful feature extraction capabilities of ResNet50, pre-trained on large-scale image datasets. This approach significantly reduced training time and computational resource requirements while maintaining high accuracy in domain-specific classification tasks.

The model was trained and evaluated on a well-curated and annotated dataset containing five distinct classes of wheat leaf conditions, ensuring robustness and reliability in disease classification.

9.2 CONCLUSION:

This project successfully developed and implemented a comprehensive wheat disease prediction system that addresses critical challenges in agricultural disease management. The system demonstrates significant achievements across multiple dimensions, establishing a robust foundation for automated plant disease identification and management.

Primary Accomplishments:

The project successfully created an end-to-end wheat disease prediction system capable of accurately identifying four major wheat disease categories: Fusarium Head Blight, Healthy Wheat, Leaf Rust, and Tan Spot. The system achieved an overall accuracy of 96.75% on a comprehensive test dataset of 1694 images, demonstrating reliable performance for practical agricultural applications.

The implementation of transfer learning using ResNet50 proved highly effective, enabling rapid model development while maintaining high classification accuracy. The pre-trained architecture provided robust feature extraction capabilities that were successfully adapted to the specific requirements of wheat disease classification, reducing training time and computational requirements compared to training from scratch.

Technical Innovations:

The system incorporates several technical innovations that enhance its practical utility and reliability. The confidence scoring mechanism provides users with uncertainty estimates that correlate strongly with prediction accuracy, enabling informed decision-making in agricultural management. The web-based interface ensures accessibility across different user skill levels and technical environments. The modular architecture design enables seamless integration between deep learning models, web application frameworks, and user interfaces. This design approach facilitates future enhancements and scalability while maintaining system stability and performance.

10.FUTURE SCOPE:

While the current research on wheat leaf disease detection using the ResNet50 transfer learning model has demonstrated promising results, there remain several opportunities for further enhancement and real-world deployment. The future scope of this work aims to bridge the gap between high-performance laboratory models and practical, scalable solutions for farmers and agricultural professionals. The following directions are proposed for future exploration and development:

1.Expansion of Dataset under Multicultural and Multiregional Conditions

To ensure broader applicability and robustness, the dataset can be expanded to include:

- Images from diverse agro-climatic zones and farming environments, where wheat is grown under varying lighting, soil, and atmospheric conditions.
- Real-time field images captured using mobile devices or drone cameras to reflect natural variations in leaf appearance due to lighting, angles, and occlusion.
- Diseased leaves captured at different growth stages of the wheat plant, enabling accurate detection during early, mid, and late stages of development.

2.Improvement in Generalization and Accuracy through Optimization Techniques:

The relatively lower classification performance for certain diseases, such as *Septoria*, highlights the need for further optimization. Potential improvements include:

- Hyperparameter tuning (e.g., learning rate, batch size, dropout rate) to fine-tune training efficiency and accuracy.
- Use of optimization algorithms such as grid search, random search, or Bayesian optimization to identify the best-performing configurations.
- Incorporating class-specific weighting in the loss function or adopting focal loss to address class imbalance and improve recognition of difficult-to-classify diseases.

3.Real-Time Application through Mobile and Web Platforms:

To maximize accessibility and usability, the trained model can be extended into practical platforms, including:

- **Mobile applications** using lightweight frameworks (e.g., TensorFlow Lite, ONNX) for edge devices, allowing farmers to capture leaf images and receive instant classification results.
- **Web-based interfaces** for researchers, agronomists, and cooperatives to analyze images uploaded from remote locations.
- **Multilingual support and voice-based instructions** to ensure accessibility for non-technical users in rural communities.

11. REFERENCES

- [1] S. Cheng et al., “A High Performance Wheat Disease Detection Based on Position Information,” *Plants*, vol. 12, no. 5, Mar. 2023, doi: 10.3390/plants12051191.
- [2] M. Ashraf, M. Abrar, N. Qadeer, A. A. Alshdadi, T. Sabbah, and M. A. Khan, “A Convolutional Neural Network Model for Wheat Crop Disease Prediction,” *Computers, Materials and Continua*, vol. 75, no. 2, pp. 3867–3882, 2023, doi: 10.32604/cmc.2023.035498.
- [3] V. K. Vishnoi, K. Kumar, and B. Kumar, “Plant disease detection using computational intelligence and image processing,” *Springer*, Feb. 01, 2021, doi: 10.1007/s41348-020-00368-0.
- [4] V. G. Krishnan, J. Deepa, P. V. Rao, V. Divya, and S. Kaviarasan, “An automated segmentation and classification model for banana leaf disease detection,” *J Appl Biol Biotechnol*, vol. 10, no. 1, pp. 213–220, Jan. 2022, doi: 10.7324/JABB.2021.100126.
- [5] M. Mahender, “Plant Leaf Disease Detection,” *Int J Res Appl Sci Eng Technol*, vol. 12, no. 4, pp. 663–666, Apr. 2024, doi: 10.22214/ijraset.2024.59839.
- [6] R. Roy, S. K. V, and L. Elizabeth Sunny, “Plant Leaf Disease Detection using SVM,” 2021. [Online]. Available: www.ijraset.com
- [7] Y. Borhani, J. Khoramdel, and E. Najafi, “A deep learning based approach for automated plant disease classification using vision transformer,” *Sci Rep*, vol. 12, no. 1, Dec. 2022, doi: 10.1038/s41598-022-15163-0.
- [8] P. TejaJ, K. KumarK, and A. BashaH, “An Efficient Approach for Classification and Identification of Plant Leaf Diseases Based on Deep Neural Networks,” *Int. Res. J. Eng. Technol.*, 2020. [Online]. Available: www.irjet.net
- [9] G. G. Manvi, G. K. N, G. R. Sree, K. Divyanjali, and D. K. Patil, “Plant Disease Detection,” *Int J Res Appl Sci Eng Technol*, vol. 10, no. 5, pp. 4538–4542, May 2022, doi: 10.22214/ijraset.2022.43221.
- [10] S. Machha, N. Jadhav, H. Kasar, and Prof. S. Chandak, “Crop Leaf Disease Diagnosis using Convolutional Neural Network,” *Int. J. Trend Sci. Res. Dev. (IJTSRD)*, vol. 4, no. 2, pp. 1–3, Feb. 2020. [Online]. Available: www.ijtsrd.com
- [11] M. Khalid, M. S. Sarfraz, U. Iqbal, M. U. Aftab, G. Niedbała, and H. T. Rauf, “Real-Time Plant Health Detection Using Deep Convolutional Neural Networks,” *Agriculture (Switzerland)*, vol. 13, no. 2, Feb. 2023, doi: 10.3390/agriculture13020510.
- [12] M. M. Islam et al., “DeepCrop: Deep learning-based crop disease prediction with web application,” *J Agric Food Res*, vol. 14, Dec. 2023, doi: 10.1016/j.jafr.2023.100764.